14 天学会 DeepSeek Harness

从「会聊天」到「会带 AI 干活」

写给高中毕业生的零基础学习计划 · 2026 年 11 月版

基于 DSH 桌面版 0.2.1-rc.5 ｜ 14 天 × 每天 60–90 分钟

这份计划包含：

· 14 天逐日任务（概念 → 动手 → 验收 → 常见坑 → 可复制提示词）

· 附录 A 提示词手册：10 条原则 + 12 个模板 + 好坏对照

· 附录 B Agent Team 手册：角色、工具、状态、限制、完整示例

· 附录 C 自带插件目录：5 组约 40 项功能，标注本机开关状态

· 附录 D–I：命令速查、成本清单、急救、术语、打卡表、重装清单

写给高中毕业生的零基础计划 • 2026 年 11 月版 • 基于 DSH 桌面版 0.2.1-rc.5

这份计划写给**没有编程基础**的人：每天 60–90 分钟，14 天，从"把 DSH 当成高级聊天框"到"能让 AI 自己读文件、写文档、查资料、组队干活，并交付一件拿得出手的作品"。

你不需要先学编程。DSH 是**工具**，就像学用 Word 不需要先会写小说；你负责"说清楚要什么、验收结果"，敲命令、写代码这些活交给它。

> 你之前那份《14 天 GitHub 学习计划》练的是"人和代码仓库打交道"。这份计划练的是"人和 AI 助手打交道"——2026 年最值钱的一项能力。两份计划不会冲突：第 13–14 天你会把它们合起来用。

这份计划要解决什么

不是"会聊天"，而是四件具体的事：

你会学到	14 天后能做什么	对应天数
用 DSH 干活	让它读你的文件、写报告、做 Word/Excel/PPT、发布网页	第 1–7 天、第 13–14 天
写提示词	把"帮我弄一下"变成"一次就做对"的任务说明	第 3–5 天 + 附录 A
懂 Agent Team	让 2–3 个 AI 队友分工协作，你当项目经理验收	第 8–10 天 + 附录 B
认识自带插件	知道 DSH 有哪些能力、在哪开、怎么装、装坏了怎么退	第 6、12 天 + 附录 C

14 天后你会做到（结营验收单）

- ☐ 能独立新建工作区、会话，并用一句清楚的话让 DSH 完成一件真事
- ☐ 能写出带**目标、背景、输入、约束、验收、输出格式**的提示词
- ☐ 会用 `@` 引用文件、会先让 DSH 计划再执行（`/plan`）、会给返工指令
- ☐ 有一个自己的 `AGENTS.md`，让 DSH 每次开会话都自动记住你的规矩
- ☐ 用 DSH 做过 Word / Excel / PDF 至少各一份
- ☐ 亲手组建过一次 Agent Team（至少 2 个队友 + 共享任务板），并验收
- ☐ 会用子代理（subagent）并行调研，会用 Workflow 批量处理
- ☐ 能说出 DSH 自带插件里至少 15 个是干什么的，会开关和安装

☐ 有一个发布到 GitHub 的作品（网站 / 文档 / 仓库）

☐ 看得懂 Token 用量，知道怎么省上下文、省时间、省钱

怎么用这份计划

1. **每天三段**：20 分钟读概念 → 50 分钟动手 → 10 分钟写学习笔记（让 DSH 帮你记，但要点自己写）。
2. **一天只学一个核心概念。**当天没做完不要紧，第二天先补练习，再做新内容。
3. 所有操作都在 **DSH 桌面应用**里完成；需要命令行时用 DSH 自己的 PowerShell 工具，或在 Windows 开始菜单打开 PowerShell。代码块里以 `#` 开头的是注释，不用输入。
4. 每节的 ✅ **完成标准** 都打勾再进入下一天；做不到就回看当天内容，或把报错原文复制给 DSH 问（这本身就是第 3–4 天的练习）。
5. **提示词要抄不要背**：每天给出的模板直接复制粘贴，改掉方括号里的内容即可。
6. 图标约定：💡 概念 | 🖱️ 界面操作 | ⌨️ 命令 | ✍️ 练习 | ✅ 验收 | ⚠️ 常见坑 | 📋 可复制的提示词

开始前：检查你的起点

按本机（E:\AI-Learning 工作区）的实际情况核对，缺什么补什么：

项目	本机情况	你需要做的
电脑	Windows 10/11	无
DSH	✅ 桌面版 0.2.1-rc.5，程序在 `E:\Deepseek`，界面地址 `http://127.0.0.1:13189`	第 1 天核对版本
DeepSeek 账号	✅ 已登录（聊天使用官方额度）	第 1 天检查"设置 → 账号"余额
默认模型	✅ `deepseek-flash`（推理等级可在模型菜单里调）	第 2 天学怎么切模型
工作区	✅ `E:\AI-Learning` 已作为工作区注册	第 1 天再建一个练习工作区
Git	✅ 已安装（GitHub 计划时配好）	第 5 天用它给工作区存档
GitHub	✅ 已有账号 `Maple7mi`、`gh` 已登录	第 7、14 天发布作品用
网络	需要能访问 DeepSeek 与 GitHub	搜索失败时先查网络

> 如果你换一台电脑，或以后重装系统，按**附录 I：换电脑 / 重装清单**重装，其余计划不变。

本机 DSH 的关键设定（第 1 天先记住这几条）

设定	本机情况	一句话解释
数据存哪	`C:\Users\Administrator\.dsh`	会话日志、配置、技能都在这里，删了不可恢复
会话日志	`~/.dsh/sessions/`	所有对话的持久记录，可用 `/export` 导出
权限预设	默认**完全权限**（danger-full-access，不弹审批）	学习期建议改成「工作区内修改」（见第 2 天）
内置实验功能	已启用：Agent Team、语音输入、会话检查器；自动审查已安装但默认未选	第 6、12 天会逐个认识
界面语言	中文	若你切成英文，按括号里的英文找同名入口

三个问题，先给答案

你在需求里点名要的三件事，后面都有专门的天数和附录；但先给"一句话版"，让你带着地图走路。

Q1：提示词到底怎么写?

提示词不是"咒语"，是**工作交办单。**一张合格的交办单有六格：目标（要什么）、背景（给谁、为什么）、输入（哪些文件/数据）、约束（不许做什么、什么风格、多长）、验收（怎么算完成）、输出格式（写成什么）。写清楚这六格，普通模型也能干出专业活；缺了验收和输出格式，再强的模型也只能猜。→ 第 3–5 天系统练，附录 A 有 12 个可复制模板。

Q2：什么叫 Agent Team?

普通对话是"你和一个 AI 助手"，Agent Team 是"**你带一个项目组**"：一个 Lead（主智能体，就是你在聊的这个）加上若干有名字、有职责的 teammates（队友），它们共享同一块任务板，可以互相发消息、认领任务、按依赖排队，你只负责定目标和验收。它适合"需要几个人分头干、又不能把上下文的细节全塞进一个对话"的活。→ 第 9–10 天亲手组一次，附录 B 是完整手册。

Q3：自带插件有什么内容?

DSH 的本体（dsh-base + web-app）一共几百个包，但摆在你面前的其实是四类东西：**界面功能**（会话树、右侧文件/终端/浏览器、交付卡片…）、**能力工具**（读写文件、跑 PowerShell、搜索、定时提醒、Office 文档…）、**协作与扩展**（子代理、Agent Team、Workflow、技能、MCP、插件市场）、**实验功能**（语音输入、自动审查、会话检查器…）。→ 第 6、12 天带你逛，附录 C 是分组目录。

学习路线图

阶段	天数	主题	你会得到
第一周：会用一个 AI 助手	1–7	界面 → 工作方式 → 提示词 → 长期记忆 → 插件技能 → 小项目	一套自己的 DSH 工作区 + 一个发布的作品
第二周：会指挥一群 AI	8–14	子代理 → Agent Team → Workflow → 插件进阶 → 结营项目	一次真实的团队协作 + 一份可展示的作品集

14 天总览

天数	主题	当天产出	难度
第 1 天	认识 DSH：工作区、会话、第一次真任务	一个练习工作区 + 一份目录说明	⭐
第 2 天	Agent 怎么工作：工具、权限、上下文、成本	一份"用量与成本"观察笔记	⭐
第 3 天	提示词基础：交办单六要素	3 个一次做对的任务	⭐⭐
第 4 天	提示词进阶：@文件、先计划、反问、返工	一份经过审批的计划	⭐⭐
第 5 天	让 DSH 记住你：AGENTS.md 与工作区结构	你的 AGENTS.md + git 存档	⭐⭐
第 6 天	自带插件与技能（上）：Office 三件套	Word + Excel + PDF 各一份	⭐⭐
第 7 天	周复盘 + 小项目：个人主页上线	公网可访问的网页	⭐⭐
第 8 天	子代理：把活分出去，别弄脏主对话	3 路并行调研汇总	⭐⭐
第 9 天	Agent Team（上）：Lead、队友、共享任务板	2 人小组产出的一份 FAQ	⭐⭐⭐
第 10 天	Agent Team（下）：3 人协作 + 依赖 + 验收	一本三章的学习手册	⭐⭐⭐
第 11 天	Workflow：用脚本编排批量任务	一份批量审计报告	⭐⭐⭐
第 12 天	插件进阶：市场、实验功能、自建技能	一个自己写的技能	⭐⭐⭐

天数	主题	当天产出	难度
第 13 天	结营项目（上）：选题 + 组建团队开工	项目计划 + 任务板	⭐⭐⭐
第 14 天	结营项目（下）：验收、发布、复盘	作品链接 + 复盘 + 30 天计划	⭐⭐⭐

第 1 天：认识 DSH——它不是聊天框，是"会动手的助手"

今日目标

分清"聊天 AI"和"Agent（智能体）"；认识工作区、会话、模型、权限四样东西；完成第一个有交付物的任务。

核心概念

💡 **普通聊天 AI**：你问，它答，回答完就结束。它只能"说"，不能"做"。

💡 **Agent（智能体）**：能**用工具**的 AI。它能读你电脑上的文件、写文件、运行命令、上网查资料，做完还告诉你做了什么。DSH 就是一个 Agent 的工作台。类比：聊天 AI 是电话咨询，Agent 是上门干活的师傅。

💡 **工作区（Workspace）**：磁盘上的一个项目文件夹，比如 `E:\AI-Learning`。DSH 侧边栏按工作区分组；一个工作区里可以有无数个会话。**会话按创建时的目录归属，不能搬到别的工区。**

💡 **会话（Session）**：一次任务的对话记录。可以重命名、置顶、**分支（Fork）**、**归档**；DSH 不删会话，归档只是从列表隐藏，可以恢复。

💡 **模型**：干活的大脑。本机默认 `deepseek-flash`（快、便宜），难任务可以在输入框的模型菜单里换更强的模型、调推理等级。

💡 **权限（Permissions）**：决定 DSH 能对你的电脑做什么。三档：**仅可查看**（只读，只看不改）、**工作区内修改**（只能动当前工作区，写之前弹审批）、**完全权限**（什么都能做，不弹审批）。本机默认是完全权限，方便但危险，第 2 天细讲。

动手步骤

第 1 步：逛一遍界面（10 分钟）

打开 DSH 桌面应用（或浏览器访问 `http://127.0.0.1:13189`），对照着找：

位置	是什么	你会用它做什么
左侧边栏	工作区 + 会话列表	新建/切换会话、搜索历史、归档
中间	对话区	输入任务、看工作过程、看交付卡片
右侧边栏	文件 / 终端 / 浏览器 / 文档预览	看 DSH 改了哪些文件、预览 Word/PDF
输入框下方	模型、权限、计划模式等开关	切模型、看上下文已用、开计划模式
左下角设置	通用 / 模型 / 账号 / 插件	学费用、换权限、管插件

试着做三件事：把鼠标悬停在一条会话上（出现"..."菜单）；点开右侧边栏；在输入框输入 `/`（看斜杠命令列表，`Esc` 关掉）。

第 2 步：建一个练习工作区（10 分钟）

```
# 在 PowerShell 里执行，或在 DSH 里直接让它做
New-Item -ItemType Directory -Force E:\AI-Learning\DSH练习 | Out-Null
New-Item -ItemType Directory -Force E:\AI-Learning\DSH练习\notes | Out-Null
cd E:\AI-Learning\DSH练习
git init
```

回到 DSH：左侧边栏 → **添加工作区** → 选择 `E:\AI-Learning\DSH练习`。然后在工作区行菜单里点"新建会话"。

> 这个练习工作区有自己的 `.git`，和 `E:\AI-Learning` 的仓库互不影响（父仓库会把它当成一个未跟踪目录，不影响使用）。

第 3 步：第一次任务——只读，不改文件（20 分钟）

在新会话里粘贴：

```
你是我的学习助手。工作目录是 E:\AI-Learning\DSH练习。

任务（只读，不要修改目录里已有的任何文件）：
1. 列出这个目录里的文件，最多两层子目录；
2. 用一张表格说明每个文件可能是干什么的（按文件名和内容判断）；
3. 把结果写入 notes/day01.md，第一行写生成时间；
4. 最后用一句话告诉我：这个目录现在最适合用来做什么。

验收标准：notes/day01.md 存在；表格至少 3 行；没有创建多余文件。
如果目录是空的，直接告诉我"目录为空"，不要为了完成表格而自己创建一堆文件。
```

观察三件事：

1. 它**先说计划再动手**（你会看到"准备读取文件""准备写入文件"这类状态）；
2. 它**调用了工具**——每次工具调用都是一张可展开的卡片，点开能看到具体命令和输出；
3. 结束后有**改动文件卡片**和**交付卡片**（右侧可点开预览）。

第 4 步：验收与收尾（10 分钟）

```
Get-ChildItem -Recurse E:\AI-Learning\DSH练习 | Select-Object FullName
Get-Content E:\AI-Learning\DSH练习\notes\day01.md
```

对照验收标准检查。然后回到会话列表：右键或"..."菜单 → **重命名**为「Day01 目录说明」→ **归档**。

✍️ **练习**：再开一个会话，让它把这个目录整理成"学习工作区"（建 `docs/`、`notes/`、`output/` 三个目录，各放一个 README 说明用途），验收后归档。

完成标准

- ☐ 能说出工作区和会话的区别
- ☐ 成功添加 `E:\AI-Learning\DSH练习` 为工作区，并在里面新建了会话
- ☐ `notes/day01.md` 已生成，内容符合要求（自己看过）
- ☐ 能指出界面上"工具调用卡片""改动文件卡片"在哪
- ☐ 知道三档权限分别是什么意思
- ☐ 会话已重命名并归档，侧边栏干净

常见坑

- ⚠️ **在错误的工作区开会话**：会话归属创建时的目录，发现错了就归档重开，别想着搬。
- ⚠️ **以为"归档=删除"**：归档只是隐藏，随时能在视图选项里恢复；DSH 不做删除。
- ⚠️ **提示词太短**（比如"整理一下"）：模型只能猜，结果必然跑偏。今天先体会，第 3 天系统解决。
- ⚠️ **不看工作过程只看结果**：结果错了却不知道为什么。**点开工具卡片**是新手最快的进步方式。

📋 今日提示词模板（复制改括号）：

```
你是我的[角色，例如：学习助理]。工作目录是 [路径]。

任务：[一句话说清要做什么]。
只读/可以修改：[明确]。
输入：[哪些文件或数据]。
产出：[写到哪个文件，什么格式]。
验收：[可检查的标准，2-4 条]。
如果信息不够，先问我，不要猜。
```

第 2 天：Agent 怎么工作——工具循环、权限、上下文、成本

今日目标

看懂 DSH 的"工作过程"；亲手体验权限与审批；理解上下文为什么会越用越贵，学会三个省钱的习惯。

核心概念

💡 **工具循环（Tool Loop）**：Agent 干活的节奏是"想 → 做 → 看结果 → 再想"，循环直到完成。每一步它只能做一件小事：读一个文件、跑一条命令、搜一次网页。**它看不见的东西，它就不能用**——所以你要把路径、文件、背景告诉它。

💡 **沙箱（Sandbox）**：DSH 动手时被"关在笼子里"，笼子的范围由权限决定。只读笼子最大，完全权限没有笼子。**沙箱不是杀毒软件**，它只管"能不能写"，不管"写得对不对"。

💡 **审批（Approval）**：在「工作区内修改」档位下，DSH 要写文件/跑命令前会弹卡片等你点允许。这是你的**安全检查点**：看清它要干什么再点。

💡 **上下文（Context）**：模型每次回答前，DSH 都会把"系统提示 + 全部历史 + 工具结果"重新发给模型。对话越长，每次要读的越多。类比：每句话都要把整本会议记录重读一遍。

💡 **压缩（Compaction）**：对话太长时 DSH 会自动把早期内容总结成摘要，让对话能继续。本机窗口约 100 万 token，用到约 80% 时触发自动压缩（触发线 = `min(窗口×80%, 窗口 − 输出上限 − 64K)`）；也可以手动 `/compact`。**压缩有代价**（要花一次大请求），所以最好在它触发前主动分段。

💡 **缓存与价格**：重复发的前缀命中 DeepSeek 上下文缓存，按约 2% 的价格计费。所以"新开会话"不贵，"每步都背着几十万 token"才贵。本机 `deepseek-flash` 参考价（元/百万 token，空闲时段）：缓存命中 0.02、未命中 1、输出 4；高峰时段翻倍。以"设置 → 账号"里的实际用量为准。

动手步骤

第 1 步：完整观察一次任务（15 分钟）

在 `E:\AI-Learning\DSH练习` 新开会话，粘贴：

```
工作目录 E:\AI-Learning\DSH练习。
任务：给我写一份「DSH 学习第一周计划」，要求：
- 7 天，每天一个主题，每天 60–90 分钟；
- 每天包含：目标、3 个动手步骤、1 个验收标准；
- 先写成 Markdown 文件 output/week1-plan.md；
- 写完后自己检查：7 天是否齐全、每天是否有验收标准，把检查结果贴在回复里。

约束：不要改动目录里其他文件；不要上网。
```

然后**边看边记**：它一共调用了几次工具？哪几次是"读"、哪几次是"写"？中间有没有返工？

第 2 步：亲手体验权限（15 分钟）

🖱️ 输入框下方 → **权限** → 切成「工作区内修改」。

再发一个任务：

> 在工作区根目录创建一个 hello.txt，内容写"第一条我自己批准的文件"。

你会看到审批卡片。这次**拒绝**它，观察它怎么反应；然后重新发一次，这次**允许**。

再切回「完全权限」，重复同样的任务，观察"不再弹审批"的区别。

> ⚠️ 完全权限下 DSH 可以改你电脑上任何文件。**只在明确自己知道它在干什么时使用**；日常建议「工作区内修改」。

第 3 步：看用量与上下文（10 分钟）

🖱️ 设置 → 通用设置 → 性能与用量（或右侧边栏的用量入口），找到：

- 本轮 **Token 用量**（输入/输出/缓存命中）；
- 输入框下方的「上下文已用」百分比。

回到会话里输入 `/compact`，手动压缩一次，观察"上下文已用"的变化。

第 4 步：省钱三习惯（10 分钟）

把这三条抄进 `notes/day02.md`，并各写一句"我以后会怎么做"：

1. **一个可验收的子任务一个会话**，做完就归档；
2. **别把长文贴进对话**，把文件路径给它，让它自己读（能省 50 倍）；
3. **错峰 + 按需降推理档**：高峰是工作日 9:00–12:00、14:00–18:00（价格 ×2），重活留到晚上/周末；简单任务在模型菜单里降一档。

✍️ **练习**：用 `/export` 把今天这个会话导出，解压看看里面有什么（这是你的学习证据）。

完成标准

- ☐ 能画出"想 → 做 → 看结果"的工具循环，并举一个今天看到的例子
- ☐ 亲眼见过一次审批卡片，知道拒绝/允许分别会发生什么
- ☐ 知道本机默认权限是"完全权限"，并会切换到"工作区内修改"
- ☐ 会用 `/compact`，能说出压缩的代价
- ☐ 能说出三种省钱习惯，并写进 `notes/day02.md`
- ☐ 导出了一次会话日志

常见坑

- ⚠️ **一直用完全权限**：图省事，但一次误操作可能改坏重要文件。**养成先看权限再发任务。**
- ⚠️ **让一个会话干一整天**：上下文越滚越大，越到后面越慢越贵还越笨。**按任务分段。**
- ⚠️ **把几万字日志直接粘贴**：等于把最贵的"未命中输入"买满。**给路径 + 要求切片阅读。**
- ⚠️ **以为"输出短就便宜"**：输出单价是缓存命中输入的约 **200 倍**，让它少说废话确实省钱，但更贵的是超长上下文。
- ⚠️ **忽略了审批卡片**：看一眼再点，它可能正要删文件。

📋 今日提示词模板（观察用）：

> 工作目录：[路径]。
> 任务：[具体任务]。
> 执行时请：先列出你打算做的 3–5 步，再开始；每完成一步用一句话汇报。
> 产出：[文件路径与格式]。
> 验收：[标准]。
> 完成后告诉我：你一共调用了哪些工具、改了哪些文件。

第 3 天：提示词基础——把"帮我弄一下"变成"一次做对"

今日目标

掌握交办单六要素，能判断一条提示词为什么失败、怎么改。

核心概念

💡 **提示词 = 工作交办单。**你给实习生写交办单时不会只写"把那个弄一下"；对 AI 也一样。

💡 **六要素**（记住这六个词，比背一百个技巧有用）：

要素	回答的问题	例子
目标	最终要什么	"写一篇 800 字的科普短文"
背景	给谁看、为什么做	"给高一学生看，放在学校公众号"
输入	依据什么材料	"参考 `notes/ai-notes.md`，只用里面的数据"
约束	不许做什么、什么风格	"不许用专业术语；不许编造数据；控制在 800 字"
验收	怎么算完成	"三个小标题；每个观点有例子；读完能回答'AI 为什么会犯错'"
输出格式	写成什么样	"Markdown，存到 `output/ai-intro.md`"

💡 **模型不会"读心"，只会"填空"。**你少写一格，它就用自己的想象填——这就是为什么"帮我写个总结"总会给你一堆正确的废话。

💡 **具体 > 抽象**：把"写得好一点"换成"每段不超过 3 行，用 2 个具体例子"；把"参考一下资料"换成"只参考 `docs/` 下这两个文件"。

💡 **一次只交办一件事**：一条提示词里塞 5 个任务，它会漏掉 2 个。宁可分 3 条发。

动手步骤

第 1 步：先看坏例子，再看差别（10 分钟）

在 `E:\AI-Learning\DSH练习` 新会话，先发：

> 帮我写个学习计划。

记下结果（大概率是空泛的 7 条）。**归档**这个会话，再新开一个，发：

> 工作目录 `E:\AI-Learning\DSH练习`。

```
背景：我是高中毕业生，零基础，每天能学 60 分钟，目标是 14 天学会用 DSH（一个 AI 智能体工具）。
任务：写一份「第 1 天到第 7 天」的入门学习计划。
约束：
- 每天只学 1 个主题，60 分钟以内能做完；
- 每天必须有一个"动手产出"（一个文件或一次操作），不能只是阅读；
- 用中文，不要用专业术语，必须用时加一句解释。
验收：
- 7 天齐全，每天有主题、20 分钟概念、40 分钟动手、验收标准；
- 每天都有一个能立刻执行的具体步骤（有文件路径或界面位置）。
输出：Markdown 表格 + 每天小节，写入 output/day03-plan.md。
```

对比两份结果，把差别写在 `notes/day03.md`：**具体在哪几处变好了？**

第 2 步：三要素练习——约束与格式（15 分钟）

同一个会话继续（别新开，这是同一件事的迭代）：

```
现在改一版：把每一天的"动手"改成必须 30 分钟内能完成的**最小任务**，
并且每天结尾加一句"今天最容易失败的地方是什么"。
只改这两点，其他保持不变，覆盖原文件。
```

观察它是否**只改这两点**。如果它顺手改了别处，这就是第 4 天要练的"返工话术"。

第 3 步：给输入，而不是给答案（15 分钟）

```
工作目录 E:\AI-Learning\DSH 练习。

输入：读取 notes/day03.md（今天的学习笔记）。
任务：把笔记整理成一份「提示词六要素速查表」。
约束：
- 只能用笔记里出现过的内容，不许补充新内容；
- 如果笔记里缺了某个要素的例子，在表里标"待补"。
输出：Markdown 表格，写入 output/prompt-cheatsheet.md，并在回复里列出"待补"的项。
```

这一步练的是：**给它材料，而不是让它凭空造。**

第 4 步：自检清单（10 分钟）

写完任何提示词后，逐条问自己：

- ☐ 目标是不是一句话能说清？
- ☐ 我给了它需要读的文件路径吗？
- ☐ 我有没有写"不要做什么"？
- ☐ 验收标准能不能被检查（有数字/有清单）？
- ☐ 输出写到哪里、什么格式，写清楚了吗？
- ☐ 这条提示词里是不是塞了两件以上的事？

完成标准

- ☐ 能背出六要素，并解释每格不写会怎样
- ☐ 同一任务，坏提示词与好提示词的产出差别，能说出至少 3 点
- ☐ 完成 `output/day03-plan.md` 与 `output/prompt-cheatsheet.md`
- ☐ 能说出"给材料"和"凭空写"的区别
- ☐ 自检清单 6 条全部能打勾

常见坑

- ⚠️ **形容词当要求**： "专业一点""口语化一点"——模型不知道你的标准。换成可数的东西。
- ⚠️ **忘了输出路径**：结果只在对话里，关掉就难找。**一律要求写入文件。**
- ⚠️ **一次塞多任务**：它会合并、遗漏、串味。任务多就分批。
- ⚠️ **验收标准写成"质量好"**：无法检查的标准等于没写。
- ⚠️ **提示词越写越长**：六个要素各一两句就够，长不等于好；把细节放进文件让它读。

📋 今日提示词模板（通用交办单）：

```
工作目录：[绝对路径]。

背景：[我是谁 / 给谁看 / 为什么做]。
任务：[一句话目标]。
输入：[要读哪些文件；不读什么]。
约束：[不许做什么] [风格] [长度/数量] [语言]。
验收：[可检查的标准 2–4 条]。
输出：[文件路径 + 格式]，并在回复里用 3 行说明你做了什么。
如果需要我补充信息，先问我，不要猜。
```

第 4 天：提示词进阶——@文件、先计划、会反问、会返工

今日目标

学会四种"高手动作"：用 `@` 精确投喂文件、用计划模式先审方案、让 DSH 反问你、用返工话术精准纠偏。

核心概念

💡 **`@` 引用**：在输入框输入 `@`，可以从候选菜单里选**文件、文件夹、会话或子智能体**。它的作用是把**路径作为引用**挂到这条消息里——模型看到的是"这份文件在这儿"，需要时**自己调用工具去读**。所以它省 token（不塞全文）、读到的永远是最新版本；而且 DSH 会给模型加一句约束：没读过就不许说"我看过"。

💡 **计划模式（`/plan`）**：让 DSH **只出方案不动手**。它会探索文件、写出完整计划，你审批通过后它才开始执行。适合"改动大、怕它乱来"的任务。审批时你可以直接驳回并写意见，它会改。

💡 **反问（ask_user_question）**：好的助手在动手前会问关键问题。DSH 有这个工具，会弹出选项让你点。**你可以在提示词里强制它反问**："如果信息不够，先问我，不要猜。"

💡 **返工话术**：结果不满意时，**别重发整段任务**，而是指出具体问题 + 划定修改范围 + 要求跑验收。句式：`上一轮的问题是[X]。只修 X，不要改动其他部分；修完把[验收命令/检查清单]的结果贴出来。`

💡 **会话的寿命**：同一个任务的迭代，**留在原会话**（上下文还在，不用重讲）；任务换了阶段、上下文已经很大，就**先让它写交接文件**，再新开会话。

动手步骤

第 1 步：`@` 引用练习（10 分钟）

在 `E:\AI-Learning\DSH练习` 新会话，输入框里敲 `@`，选 `notes/day03.md`（或直接输入 `@notes/day03.md`），然后接着写：

```
@notes/day03.md 这是我昨天的笔记。
任务：提取其中的"提示词六要素"，写成一张速查表，保存到 output/prompt-cheatsheet-v2.md。
约束：只提取笔记里有的内容；表格最多 6 行。
```

对比昨天的输出：用 `@` 和用文字描述材料，哪个更准？

第 2 步：用 `/plan` 审一份方案（20 分钟，今天最重要）

新开会话，输入 `/plan` 进入计划模式（输入框会出现计划模式提示），然后发：

```
工作目录 E:\AI-Learning\DSH练习。
目标：把这个目录改造成一个能长期用的学习工作区。
```

> 要求：
> - 目录结构：docs/（spec、progress、decisions）、notes/、tasks/、output/；
> - 根目录放 AGENTS.md（我之后会自己写内容，先给骨架）和 README.md；
> - 不要删除任何现有文件；已有文件只做归类，不修改内容；
> - 给我一份可以逐步执行的计划，列出每一步会动哪些文件。

它会给出一个计划卡片。**先别批准**，做三件事：

1. 读一遍计划，找一处你觉得多余或遗漏的地方；
2. 驳回，写一句意见（例如"第三步会移动我已经有的文件，改成只新建目录，不移动"）；
3. 看它修改后的计划，再批准。

批准后它才会真正动手。结束后用 `Get-ChildItem -Recurse` 核对结构和计划是否一致。

第 3 步：让 DSH 学会反问（10 分钟）

新会话，发一个**故意含糊**的需求：

> 帮我做一个个人网站。

观察：它是直接开始编，还是先问你？然后归档这个会话，重开，用这个模板：

> 我想做一个个人网站，但还没想清楚细节。
> 要求：动手前先问我 3 个最关键的问题（用选项形式），等我回答后再给方案。
> 不要开始写代码。

体验一下"被选项问住"的感觉——这就是好助手该有的样子。

第 4 步：返工话术练习（10 分钟）

回到第 2 步的会话，让它做一件小事：

> 在 README.md 里写一段 120 字的工作区简介，用中文。

收到结果后，挑一个真实问题（比如太长、太正式、没提到用途），用固定句式返工：

> 上一轮的问题：简介有 200 多字且没提"给谁用"。
> 只修这两点：压到 120 字以内；第二句说明这个工作区是给"零基础学习者"用的。
> 不要改动 README 的其他部分。改完把字数统计贴出来。

看它是否**只改这一段**。如果它又顺手改了别处，再强调一次"只改这一段"，并检查 diff。

第 5 步：交接与换会话（5 分钟）

在会话里发：

> 把当前状态写进 docs/handoff.md：已完成 / 未完成 / 下一步 / 关键文件路径 / 已知问题。
> 写完就结束，不要继续做别的。

然后**归档**这个会话，新开一个，发：

> 读 docs/handoff.md，告诉我下一步该做什么，然后等我确认。

体会：**文件是记忆，会话只是过程。**

完成标准

- ☐ 会用 `@` 引用文件，并说出它比复制粘贴好在哪
- ☐ 完整走完一次"计划 → 驳回 → 修改 → 批准 → 执行"
- ☐ 见过一次 DSH 主动反问（ask_user_question）
- ☐ 用标准返工句式修正过一次结果，且只改了指定部分
- ☐ 写过一次 `docs/handoff.md`，并在新会话里成功接上
- ☐ 能说出"什么情况该留在原会话、什么情况该新开"

常见坑

- ⚠️ **计划模式里还催它"快点做"**：计划没批准它不会动手，催也没用，先审批。
- ⚠️ **返工时重发整段原任务**：等于让它重新猜，之前的上下文白费。**只指出问题。**
- ⚠️ **把 `@` 当装饰**：引用了一堆无关文件，反而干扰。只挂真正要用的 1–3 个。
- ⚠️ **驳回意见写"再改改"**：它不知道改哪。写具体到段落/行。
- ⚠️ **以为新会话=重来**：新会话没有你的历史，**交接文件**是桥。

📋 今日提示词模板（计划模式）：

```
/plan

工作目录：[路径]。
目标：[要达成的结果]。
现状：[已知的文件/结构/限制]。
要求：[步骤约束，例如"不删除任何文件""只新建不改旧"]。
请先探索再给计划，计划里逐步列出：做什么、动哪些文件、怎么验证。
不要动手，等我批准。
```

📋 返工模板：

```
上一轮的问题：[1–3 条具体问题，最好带位置]。
只修这些，不要扩散改动。
改完做：[验收检查]，把结果贴出来。
```

第 5 天：让 DSH 记住你——AGENTS.md 与工作区结构

今日目标

给 DSH 建一套"员工手册"，让它每次开新会话都自动知道你的规矩；学会用文件而不是聊天记录来管理长期信息。

核心概念

💡 **AGENTS.md = 员工手册。**它放在你的项目里，**每次会话的第一个请求都会自动被读进去。**写"长期不变的规则"，不要写"今天的任务"。

💡 **加载规则（很重要）**：DSH 会按顺序找——

1. `~/.dsh/AGENTS.md`（你的全局规则，所有项目生效）
2. `~/.agents/AGENTS.md`（共享配置）
3. 从**项目根**到**会话工作目录**每一层的 `AGENTS.md`、`CLAUDE.md`、`AGENTS.local.md`、`CLAUDE.local.md`

项目根 = 最近的包含 `.git` 的上级目录；如果一路找不到 `.git`，项目根就等于会话工作目录（也就是说，父目录的 AGENTS.md 不会被加载）。总预算 64KB，越具体的越优先。

💡 `docs/` **三件套**：`spec.md`（需求与设计，唯一真源）、`progress.md`（进度板：做到哪、下一步、已知问题）、`decisions.md`（关键决策 + 理由）。它们回答"这个项目现在是什么状态"，比翻聊天记录可靠。

💡 `tasks/`：一个子任务一张单（目标/输入/产出/验收清单/结果）。**任务单让你和 DSH 都不用猜。**

💡 **文件是长期记忆，聊天是工作过程。**DSH 的会话可能归档、可能压缩、可能很长；文件不会。**重要结论一定落文件。**

动手步骤

第 1 步：建项目骨架（10 分钟）

在 `E:\AI-Learning\DSH练习` 新开会话，用计划模式（或直接）发：

```
工作目录 E:\AI-Learning\DSH练习。

任务：按下面的结构建好目录和空文件模板，已有文件不删不改：
- AGENTS.md（先写骨架，我随后自己补内容）
- README.md（一句话说明这个工作区）
- docs/spec.md、docs/progress.md、docs/decisions.md
- tasks/T01-示例任务.md
每个文件写 3–8 行模板：这个文件是干什么的、怎么写、什么时候更新。
验收：文件都存在；每份模板里都有"填写说明"。
不要上网，不要写我的个人信息。
```

第 2 步：写你自己的 AGENTS.md（20 分钟，今天核心）

把下面模板改成你的实际情况（改方括号内容），写进 `E:\AI-Learning\DSH练习\AGENTS.md`：

```
# 我的 DSH 工作区规则

## 我是谁
- 高中毕业生，非计算机专业，正在用 DSH 学习 AI 工具的用法。
- 我的目标是：用 DSH 完成学习任务、产出一份能拿得出手的作品。

## 怎么和我说话
- 用中文，先给结论再给细节。
- 不要用没解释的专业术语；必须用时括号里解释。
- 一次只做一件事，做完告诉我改了什么文件。
- 不确定就问我，不要猜（用选项形式问）。

## 工作约定
- 所有产出写进 `output/`；过程记录写进 `notes/`。
- 修改文件前先读再改；不要删除我已有的文件。
- 长任务先给计划，我批准后再动手。
- 每完成一项，把结果写进 `docs/progress.md`。

## 红线
- 不要访问网络上传我的任何个人信息。
- 不要动 `C:\` 下其他目录，只在这个工作区内工作。
- 不要编造数据；找不到就说找不到。

## 验收习惯
- 交付时附：改了哪些文件、怎么验证、还剩什么问题。
- 文档类产出：Markdown 存一份，需要时再导 Word/PDF。
```

第 3 步：验证规则真的生效（10 分钟）

新开一个会话，发：

```
不要做别的。读你的工作区规则，用三条告诉我：你最应该注意的红线是什么？
```

如果它能准确复述你的红线，说明 AGENTS.md 生效了。再做一次"坏规则"实验：在 AGENTS.md 末尾加一行 `- 每次回答最后都加一句"继续加油"。`，新会话随便问个问题，看它是否照做——**这证明了它每次都会读这份文件**。验证完删掉这行。

第 4 步：用 docs/ 管理进度（10 分钟）

在同一个会话里发：

```
更新 docs/progress.md 和 docs/decisions.md：
- progress：写下今天完成的事、明天要做的、已知问题；
- decisions：写下"为什么用 AGENTS.md + docs 的结构"这个决定，以及理由。
每份不超过 15 行。
```

第 5 步：git 存档（5 分钟）

```
cd E:\AI-Learning\DSH练习
git add -A
git commit -m "docs: 建立 DSH 学习工作区结构"
git log --oneline -3
```

> 这也是给以后做铺垫：GitHub 作品集、换电脑迁移，都靠它。

完成标准

- ☐ `E:\AI-Learning\DSH练习` 是 git 仓库，有第一次提交
- ☐ AGENTS.md 写好了"我是谁 / 怎么说话 / 工作约定 / 红线 / 验收"五部分
- ☐ 新会话能复述你的红线（证明加载生效）
- ☐ 做过"加一条规则 → 新会话验证 → 删掉"的实验
- ☐ `docs/progress.md`、`docs/decisions.md` 已写，且不是空模板
- ☐ 能说清"项目根"和"AGENTS.md 加载顺序"

常见坑

- ⚠️ **把任务写进 AGENTS.md**：它每次请求都会带上，越写越长、越来越慢。**任务进 tasks/，规则进 AGENTS.md。**
- ⚠️ **项目没 `git init`**：AGENTS.md 的层级链、项目级技能扫描都会失效或只认当前目录。**开工先 git init。**
- ⚠️ **频繁改 AGENTS.md**：会让缓存从改动处失效，也会让模型对规则困惑。想好再写。
- ⚠️ **docs/ 写完就不更新**：进度板的价值在于"随时是最新的"，每次收尾花 2 分钟更新。
- ⚠️ **规则写得太抽象**（"要专业"）：写可执行的，例如"先结论后细节""每段不超过 3 行"。

📋 今日提示词模板（项目骨架）：

```
工作目录：[路径]。
任务：按以下结构创建项目骨架（不删不改已有文件）：
[目录树]
每个文件写 3-8 行模板 + "填写说明"。
验收：文件齐全、模板可读、没有动其他文件。
完成后列出所有新建文件。
```

第 6 天：自带插件与技能（上）——先会用，再会谈

今日目标

分清"插件"和"技能"；找到 DSH 的插件入口；用内置 Office 技能做出 Word / Excel / PDF 三份专业交付物。

核心概念

💡 **插件（Plugin）= 装进 DSH 的功能包。**聊天界面、文件树、终端、网页搜索、定时提醒、Agent Team……都是插件。开关在**设置 → 插件 → 内置插件（只读清单）**和**侧边栏插件页**里；"插件市场（Discover）"可以装第三方的，但本机默认没启用（第 12 天再开）。

💡 **技能（Skill）= 写在文件夹里的操作规程。**一个技能就是一个目录 + 一个 `SKILL.md`，里面写"这类活该怎么干"。和提示词不同：**提示词每次都发，技能只在需要时加载**——所以技能是"随身工具书"，省 token 又专业。

💡 `/skill` **与技能工具**：模型发现任务需要某技能时，会调用技能工具把它读进来。你输入 `/skill` 也能查看/加载技能。

💡 **内置 Office 技能**：DSH 自带 Word / PowerPoint / Excel 的处理能力（底层是 python-docx、python-pptx、openpyxl 和 LibreOffice），能做正式文档、报表、PPT，还能导出 PDF。**你不需要写代码，描述要什么就行。**

💡 **MCP**：一种"外接工具协议"，可以用 MCP 把外部服务（数据库、浏览器、公司内部系统）接进 DSH。知道有这回事即可，第 12 天再看。

动手步骤

第 1 步：逛插件（10 分钟）

🖱️ 三处都找一下：

1. **设置 → 插件 → 内置插件**：只读清单，按预设分组显示当前加载了哪些；
2. **侧边栏 → 插件页**：真正能操作的地方——安装、启用/禁用组合包、开关某一行、打开官方配置页；
3. **插件页 → 实验性插件分组**：语音输入、会话检查器、Agent Team 等实验功能都在这里。

记下 5 个你认识的功能（例如：文件树、终端、网页搜索、语音输入、会话检查器），写进 `notes/day06.md`。

> ⚠️ 看到几十上百行插件名不要慌：本机安装包里约有 306 个包，但你在界面上能感知的功能大约只有 40 项。**附录 C 按用途分了组**。另外本机**没有"发现"标签页**（插件市场 bundle 没启用），这是正常的。

第 2 步：开一个 Office 任务（25 分钟，今天核心）

在 `E:\AI-Learning\DSH练习` 新会话，粘贴：

```
工作目录 E:\AI-Learning\DSH练习。
请用你的 Office 文档能力，帮我做三份文件，全部放到 output/ 下：

1) output/简历.docx：一份高中生简历模板（我的信息先用[姓名]占位）。
   结构：基本信息 / 教育经历 / 技能 / 项目与作品 / 自我评价。
   要求：A4，正文用小四号，标题加粗；只做模板，不要编造我的经历。

2) output/学习计划.xlsx：一个 14 天学习计划表。
   列：天数 / 主题 / 今日产出 / 完成情况（下拉：未开始/进行中/已完成）/ 备注。
   要求：表头加粗、冻结首行、列宽合适、加一个完成率公式。

3) output/DSH使用说明.pdf：把 AGENTS.md 和 docs/spec.md 的内容整理成一份 2 页以内的
   使用说明，导出 PDF。

验收：三个文件都存在且能正常打开；xlsx 有完成率公式；PDF 页数 ≤ 2。
做完后告诉我每个文件的大小和页数/行数。
```

第 3 步：检查交付物（10 分钟）

🖱️ 右侧边栏 → 文件树 → 点开三个文件预览（Word/Excel/PDF 都能直接预览）。

用 PowerShell 核对：

```
Get-ChildItem E:\AI-Learning\DSH练习\output | Select-Object Name,Length,LastWriteTime
```

发现问题就用**返工话术**（第 4 天学的）修，不要重发整段。

第 4 步：认识技能（10 分钟）

在会话里发：

```
列出你现在可以用的技能（skill），每个用一句话说明什么时候该用。
```

再输入 `/skill` 看命令面板。观察：**技能是按需加载的**——它平时不占上下文，用到才读。

✍️ **练习**：让 DSH 写一个 `notes/skills-notes.md`，记录"插件 vs 技能"的区别和你今天见到的 3 个技能。

完成标准

- ☐ 找到设置里的内置插件清单、侧边栏插件页、实验性分组三个入口，并知道插件市场默认没开
- ☐ 能说出插件和技能的三点区别（谁装、谁写、什么时候加载）

- ☐ Word 简历、Excel 计划表、PDF 说明三份文件都生成并打开检查过
- ☐ Excel 里有公式且能算出结果
- ☐ 用 `/skill` 看过技能列表
- ☐ 知道 Office 能力是"内置技能"，不需要自己装 Python 库

常见坑

- ⚠️ **在插件页乱关东西**：关掉基础插件可能导致界面功能消失。**不确定的不要动**；动了出问题，按附录 F 回退。
- ⚠️ **以为"技能=插件"**：插件是功能，技能是流程；写法也完全不同（附录 C 有对照）。
- ⚠️ **文档要求太笼统**（"做个好看的简历"）：给结构、字号、页数，它才能交付可用的东西。
- ⚠️ **忘了让它存文件**：只在对话里展示的文档，等于没做。
- ⚠️ **中文字体/乱码**：让它在生成后"打开检查有没有乱码"，并说明用系统字体。

📋 今日提示词模板（文档三件套）：

```
工作目录：[路径]。
用 Office/文档技能生成以下文件（全部放 output/）：
1）[文件名.docx]：[结构、页数、字号要求]
2）[文件名.xlsx]：[列名、公式、格式要求]
3）[文件名.pdf]：[来源材料、页数上限]
验收：[每个文件的检查点]。
生成后自查：能否打开、中文是否正常、公式是否算出结果。
```

第 7 天：周复盘 + 小项目——把主页发到公网

今日目标

把第一周的知识串起来，用 DSH 独立完成一个小项目并发布到 GitHub Pages；同时做一次周自测。

动手步骤

第 1 步：周自测（15 分钟，先自己答再对照）

1. 工作区和会话的区别是什么？会话能搬到别的工作区吗？
2. DSH 干活的"工具循环"是哪四步？
3. 三档权限分别是什么？本机默认哪一档？
4. 上下文压缩是什么？为什么要主动分段？
5. 提示词六要素是哪六个？
6. `@` 引用和复制粘贴相比，好在哪？
7. 计划模式的"批准"意味着什么？
8. AGENTS.md 的加载顺序是什么？"项目根"怎么确定？
9. 插件和技能的区别？
10. 请一天假没学，第二天应该先做什么、后做什么？

答案在第 1–6 天正文里，**至少答对 8 题再进第二周。**

第 2 步：小项目——个人主页（35 分钟）

用第 4 天的计划模式，在 `E:\AI-Learning\DSH练习` 里做：

```
/plan

工作目录 E:\AI-Learning\DSH练习。
目标：做一个可以发布到 GitHub Pages 的个人主页，文件直接放在工作区根目录。

要求：
- index.html：单页，包含 标题、自我介绍、我在学什么（至少 3 项）、
  14 天学习进度表、我的作品链接区（先占位）；
- 样式写在 style.css，简洁、手机能看（响应式）；
- 不要用任何外部 CDN（国内可能打不开），字体用系统字体；
- 移动端可读：字号不小于 14px，正文宽度不超过 760px；
- 内容用中文，语气真诚，不要吹牛。

请先给计划（列出会创建哪些文件、每个文件干什么），我批准后再写。
```

> 为什么放根目录？因为 GitHub Pages 的"从分支发布"只支持**仓库根目录**或 `/docs` 目录，不支持任意子目录。主页放根目录，发布最省事。

批准后让它执行。完成后：

🖱️ 右侧边栏 → 文件树 → 点开 `index.html` 预览；或用浏览器打开本地文件。

第 3 步：用 DSH 自查 + 返工（15 分钟）

```
请检查主页文件：
1. HTML 里引用的 CSS 路径是否正确；
2. 手机上是否会出现横向滚动（给出你的判断依据）；
3. 进度表是否 14 行齐全。
只修发现的问题，修完把检查结果逐条告诉我。
```

第 4 步：发布到 GitHub Pages（15 分钟）

```
cd E:\AI-Learning\DSH练习
git add -A
git commit -m "feat: 完成个人主页初版"

# 建远程仓库并推送（本机 gh 已登录）
gh repo create dsh-site --public --source . --remote origin --push
```

然后网页操作：仓库 → **Settings** → **Pages** → Source 选 `Deploy from a branch` → 分支 `main`、目录 `/(root)` → Save。等 1–3 分钟，访问 `https://Maple7mi.github.io/dsh-site/`。

> 也可以直接让 DSH 用 `gh` 和它的 GitHub 知识帮你走完；但页面设置这一步，推荐你亲手点一遍。

第 5 步：周复盘（10 分钟）

写 `docs/progress.md` 和一份 `notes/week1-review.md`：

- 本周最大的 3 个收获；
- 最卡的 2 个地方 + 怎么解决的；
- 下周最想学会的 1 件事；
- 主页链接（贴上去，给自己看）。

完成标准

- ☐ 周自测 10 题至少答对 8 题
- ☐ `index.html` + `style.css` 写完，桌面和手机都看过
- ☐ 14 天进度表在页面上正常显示
- ☐ 仓库已推到 GitHub，Pages 地址能打开
- ☐ 用手机流量打开一次（确认真公网可访问）
- ☐ 写完第一周复盘

常见坑

- ⚠️ **Pages 404**：Source 要选 `main` + `/ (root)`（或把主页放进 `/docs`）；刚开启要等 1–3 分钟。
- ⚠️ **用了外部字体/CDN**：国内打不开就是白屏，**只用系统字体和本地文件。**
- ⚠️ **忘了** `.gitignore`：把临时文件、截图、`node_modules` 排除掉。
- ⚠️ **把隐私写进主页**：这是全世界可见的，别放身份证、住址、手机号。
- ⚠️ **自测不会就跳过**：第二周的 Agent Team 建立在第一周的概念上，**先补再走。**

第 8 天：子代理——把活分出去，别弄脏主对话

今日目标

理解"上下文隔离"为什么重要；会用子代理（subagent）并行做独立调研；知道它和 Agent Team 的区别。

核心概念

💡 **子代理（subagent）= 临时叫来的外援。**主对话把一件事交给它，它**自己开一个干净的上下文**去干，干完只把结果交回来。中间读了多少文件、试错了几次，**不会污染主对话。**

💡 **两种子代理**：

- **spawn（新建）**：给它一份任务说明，它从零开始，不看你之前聊过什么。适合"独立调研、独立实现"。
- **fork（继承）**：它继承你**已完成的对话**，适合"接着我刚才的思路往下做"。注意：fork 会把上下文一起带过去，更贵。

💡 **一次性 / 可继续**：子代理跑完会给你结果；如果还想要更多，可以再给它发消息继续（在右侧的"子智能体"面板里能看到每次的会话记录）。

💡 **什么时候用**：任务可以**独立完成 + 结论可压缩成一段话。**例如：读 5 个文件各写摘要、并行查 3 个主题、让另一个"人"审查你写的文档。

💡 **什么时候不用**：任务很小（自己两分钟就做了）、强依赖中间过程（每一步都要你拍板）、需要频繁改同一批文件。

动手步骤

第 1 步：三路并行调研（20 分钟）

在 `E:\AI-Learning\DSH练习` 新会话，粘贴：

```
工作目录 E:\AI-Learning\DSH练习。
请用子代理并行做三路调研，等三路都回来后再汇总。

任务 A（子代理）：读 E:\AI-Learning\GitHub学习\14天GitHub学习计划.md 的第 1–700 行，
提炼"第一周的 7 个主题"和"3 条可迁移到 AI 学习的方法"，写成 200 字以内的要点。

任务 B（子代理）：读 E:\AI-Learning\DeepSeek-工作区整理指南.md 全文，
提炼"控制上下文与成本的 5 个杠杆"，每条一句话。

任务 C（子代理）：读 E:\AI-Learning\工作区模板\AGENTS.md、docs\spec.md、docs\progress.md、tasks\T01-示例任务.md，
```

```
总结这套模板的分工方式，200 字以内。

汇总要求：把 A/B/C 三份结果合并成 output/research/week2-inputs.md，
结构：三节 + 最后一节「我打算怎么用」，由你（主对话）来写，不要编造 A/B/C 没说的内容。

验收：三份子结果各有出处；汇总文件存在；没有把原文大段复制进来。
```

观察：

1. 主对话里出现"准备协调子智能体"之类的状态；
2. 右侧/会话里的**子智能体**面板能看到 3 个独立会话；
3. 子代理之间互不干扰，主对话只收到摘要。

第 2 步：比较"有子代理"和"没子代理"（10 分钟）

把同样的任务 A 直接在**主会话**里做一遍（读那 700 行）。对比：

- 主会话的"上下文已用"涨了多少？
- 结果质量差多少？

写进 `notes/day08.md`：什么时候值得为"上下文干净"多付一次请求的钱。

第 3 步：fork 体验（10 分钟）

在当前会话（已经聊了不少）发：

```
用 subagent_fork 起一个继承当前对话的子代理，让它：
1. 读我们刚才的 output/research/week2-inputs.md；
2. 站在"读者"角度挑 3 个最可能看不懂的地方；
3. 给修改建议，写到 output/research/review.md。
```

观察它是否"知道我们刚才干了什么"（这就是 fork 的价值）。

第 4 步：看成本（5 分钟）

对比本轮 Token 用量：3 个 spawn 子代理 vs 1 个 fork。记下数字（`notes/day08.md`）。

完成标准

- ☐ 能说出子代理解决的三个问题：上下文隔离、并行、专业化
- ☐ 成功让 3 个子代理并行调研并汇总
- ☐ 见过"子智能体"面板，知道每个子代理是独立会话
- ☐ 体验过 fork 与 spawn 的区别
- ☐ 能说出"什么时候不该用子代理"

常见坑

- ⚠️ **把需要你拍板的活交给子代理**：它自己决定，回来可能整个方向都错了。**决策留在主对话。**
- ⚠️ **子任务描述太粗**（"调研一下"）：子代理看不到主对话，必须**自包含**地写清目标/输入/输出/验收。
- ⚠️ **让子代理改同一批文件**：互相覆盖。**一个子代理一个文件/目录。**
- ⚠️ **滥用 fork**：fork 会继承上下文，比 spawn 贵；能用 spawn 说清楚的就别 fork。
- ⚠️ **忘了回收结果**：子代理的产出可能在它自己的会话里，**要求它写文件**，别只让它回复。

📋 今日提示词模板（并行调研）：

```
工作目录：[路径]。
用 [N] 个子代理并行完成以下调研，等全部回来再汇总：
- 任务 A：[目标/读什么/产出什么/字数或格式]
- 任务 B：...
- 任务 C：...
每个子代理把结果写到 output/research/[名称].md。
汇总：合并成一份报告，标注每节来源；不要编造。
```

第 9 天：Agent Team（上）——什么叫"带一支 AI 项目组"

今日目标

搞清楚 Agent Team 是什么、由哪些角色组成；亲手让 DSH 组建一个 2 人小组，完成一份有依赖关系的产出；学会看"智能体团队"面板。

核心概念

💡 **Agent Team = 一个 Lead + 若干具名队友。**你现在聊天的这个会话就是 **Lead（主智能体）**。它可以创建队友（teammate），队友有自己的名字（例如 `researcher`、`writer`）、职责和**独立的持久会话。**

💡 **和子代理的区别**（这是今天最重要的理解）：

	子代理 subagent	**Agent Team**
生命周期	一次任务，回来就结束	**持久**：可被唤醒、可继续干、可跨多轮
通信	只跟主对话	**成员之间可以互相发消息**，也能找 Lead
任务管理	没有	**共享任务板**：任务、依赖、认领、完成
适合	独立、一次性的活	多人分工、要协调、要反复沟通的活

💡 **共享任务板（Team Task Board）**：团队的中心。每个任务有标题、详情、**依赖（blocked_by）**、**写入范围（write_scopes）**、负责人（owner）。只有依赖都完成，任务才"可开始"；成员认领后才算"进行中"；完成、释放、重开都要改状态。

💡 **写入范围（write_scopes）只是提醒**：它标出"这个任务预计改哪些文件"，两个进行中的任务范围重叠时会提醒，但**不会阻止**任何操作。真正的安全靠你分配任务时**一个文件只给一个人。**

💡 **成员状态**：`running`（正在干）、`inactive`（没在跑，不等于干完了）、`provisioning`（正在创建）、`failed`（创建失败）。**inactive 不代表任务完成**——要看任务板和它的产出。

💡 **只有 Lead 能做三件事**：创建队友、指派/转派任务、中断队友。队友之间可以互相发消息，但不能建人。

💡 **本机限制**：最多 8 个队友；任务板最多 256 个活动任务；单条消息最大 64KB；**所有成员共享同一个目录**（没有隔离、没有文件锁、没有自动合并）；**不支持嵌套团队**；名字永久不复用；**任务 owner 不会自动释放**（队友挂了任务还挂在它名下，需 Lead 改派）。

动手步骤

第 1 步：观察一次完整协作（25 分钟，今天核心）

在 `E:\AI-Learning\DSH练习` 新会话，粘贴：

> 工作目录 E:\AI-Learning\DSH练习。
> 我要你用 Agent Team（智能体团队）完成一件小事，我要观察整个协作过程。
>
> 目标：产出一份「DSH 新手常见问题 FAQ」，共 10 个问题，写到 output/faq.md。
>
> 要求：
> 1. 创建 2 个队友：
> - researcher：负责收集和整理问题素材（基于 E:\AI-Learning 下的现有文档，不许编造）；
> - writer：负责把素材写成 FAQ 正文。
> 2. 在共享任务板上创建两个任务：
> - 任务 1「整理 FAQ 素材」，交给 researcher，写入范围 notes/faq-source.md；
> - 任务 2「撰写 FAQ」，交给 writer，写入范围 output/faq.md，依赖任务 1。
> 3. 队友都是 fresh 启动（不带我的对话历史），所以你的任务描述要自包含：写清楚读哪些文件、产出什么、验收标准。
> 4. 你自己负责：
> - 先给每个队友发一条完整任务消息再让它们开工；
> - 用 list_agents 看状态，非必要时不要反复轮询；
> - 最后验收：10 个问题、每个有答案、术语解释过、没有编造，不合格就返工；
> - 验收通过后，把任务板状态和成员状态汇报给我。

第 2 步：在界面上观察团队（10 分钟）

找到**智能体团队**面板（会话里的团队入口/右侧面板），对照看：

- **成员**：lead + researcher + writer，各自状态；
- **共享任务**：两个任务、依赖关系、owner、"可开始 / 被依赖阻塞 / 进行中 / 已完成"；
- **写入范围**：任务上标注的路径；
- **打开成员会话**：点进去看队友到底干了什么（这是最好的学习材料）。

记录 3 个观察，写进 `notes/day09.md`。

第 3 步：故意制造一次"依赖阻塞"（10 分钟）

在同一会话追加：

> 再建一个任务：「给 FAQ 加一个 3 题的快速自测」，依赖「撰写 FAQ」。
> 先别指派，让我看看它是不是显示"被依赖阻塞"。

观察任务板状态。等 writer 完成后，再让它把任务派给 writer。

第 4 步：理解等待与唤醒（5 分钟）

问 Lead（用普通对话）：

> 解释一下你刚才等队友时用的 wait_agent 是做什么的；如果队友已经 inactive，你会怎么办？

正确答案应包含：wait_agent 只观察**调用之后**发生的变化（状态/message/任务变更），**不会唤醒**成员；没有活跃成员时立即返回 noProgress；此时应该先用 list_agents 看状态，再用 send_message 唤醒需要的队友，**不要空等。**

完成标准

- ☐ 能用自己的话解释 Lead / 队友 / 共享任务板 / 写入范围 / 依赖
- ☐ 成功让 DSH 创建 2 个队友并完成 FAQ
- ☐ 在界面上看过成员状态和任务板状态
- ☐ 点开过至少一个队友的会话，看它实际怎么干活
- ☐ 知道"inactive ≠ 完成"、"写入范围只是提醒"、"owner 不会自动释放"
- ☐ 能说出 3 个 Agent Team 适合的任务、2 个不适合的

常见坑

- ⚠️ **把"建团队"当成万能药**：3 人团队的开销约为单独做的 2–4 倍（每个队友都在独立烧 token）。**小任务别开团。**
- ⚠️ **队友任务写得像"你懂的"**：fresh 队友没有你的历史，**必须自包含。**
- ⚠️ **两个人改同一个文件**：写入范围只提醒不阻止，最后会互相覆盖。**按文件/目录拆清楚。**
- ⚠️ **Lead 空等**：没有活跃队友时 wait_agent 会立刻返回；要先 list_agents 再 send_message。
- ⚠️ **以为队友干完会自动通知你**：Lead 要主动 wait_agent + 重新 list 任务，再验收。
- ⚠️ **没验收就汇报**：Lead 的最终答案要对结果负责，**先自己读产出。**

📋 今日提示词模板（组建小团队）：

```
工作目录：[路径]。
用 Agent Team 完成：[一句话目标]，产出：[文件路径]。

请这样做：
1. 创建 [N] 个队友：[名字1]（职责[..]）、[名字2]（职责[..]）；
2. 建共享任务：[任务 1 描述/写入范围/验收]、[任务 2 描述/依赖任务 1/写入范围/验收]；
3. 每个队友的任务消息要自包含（它们是 fresh 启动，看不到我们的对话）；
4. 你来盯进度、验收、返工；完成后汇报任务板和成员状态。
限制：两个队友不许写同一个文件。
```

第 10 天：Agent Team（下）——3 人协作实战

今日目标

完成一次接近真实项目的团队协作：多任务、有依赖、有审查、有 Lead 验收；同时学会控制成本和风险。

核心概念

💡 **分工三种经典模式**：

- **流水线**：调研 → 写作 → 审查（任务按依赖串起来）；
- **分片**：三章各一人写（同一类任务拆成互不重叠的片）；
- **红队**：一个人专门挑错（reviewer），逼出质量问题。

💡 **Lead 的三项硬职责**：**拆得动**（任务能独立验收）、**说得清**（每个任务自包含）、**验得准**（按验收标准逐条检查）。队友干得好不好，八成取决于 Lead 的任务书写得好不好。

💡 **成本会放大**：3 个队友 + 1 个审查 = 至少 5 条独立的模型运行线。**先小后大**：第一次跑 2 个队友，熟练了再上 3–4 个。

动手步骤

第 1 步：3 人项目开工（35 分钟）

在 `E:\AI-Learning\DSH练习` 新开会话，粘贴（这是今天的主任务）：

```
工作目录 E:\AI-Learning\DSH练习。
用 Agent Team 完成一个 3 人项目，产出《我的 DSH 14 天学习手册》。

目标文件结构：
- output/handbook/ch1.md：认识 DSH（工作区/会话/权限/模型/上下文）
- output/handbook/ch2.md：提示词方法（六要素 + 3 个模板 + 3 个反例）
- output/handbook/ch3.md：Agent Team 怎么用（概念/步骤/限制/3 个坑）
- output/handbook/review.md：审查问题清单
- output/handbook/README.md：目录 + 三章摘要（由你最后合成）

要求：
1. 创建 4 个队友：writer-a、writer-b、writer-c、reviewer；
2. 任务板：
   - 任务 1/2/3：三章写作，分别给 writer-a/b/c，各自只写自己的文件；
   - 任务 4：审查三章，给 reviewer，blocked_by 任务 1、2、3；
   - 任务 5：合成 README，给你自己（Lead），blocked_by 任务 4。
3. 每章验收标准写进任务：800–1200 字、含 1 个表格、含 3 条常见坑、术语有解释、中文。
4. 每章素材来源：E:\AI-Learning 下的现有文档（14天GitHub学习计划、工作区整理指南、工作区模板），不许编造。
```

```
5. reviewer 的产出是"问题清单 + 修改建议"，不要直接改别人的文件；修改由对应 writer 返工。
6. 全程你来协调：谁 inactive 就 send_message 唤醒；不要在没活跃队友时干等。
7. 最后汇报：用了多久、任务板变更了几次、reviewer 挑出几条问题、最终文件清单。

限制：
- 四个队友写不同文件，禁止交叉修改；
- 如果超过 8 个任务或出现写入范围重叠，先停下来告诉我。
```

第 2 步：当"监工"而不是"甩手掌柜"（10 分钟）

在它跑的过程中，做三件事：

1. 每 2–3 分钟在对话里问一句"现在进度如何？谁在跑？"（**不要频繁打断**）；
2. 觉得某个方向不对，立刻用 `send_message` 给对应队友纠偏（例如"第 2 章别写太长，控制在 1200 字内"）；
3. 发现某队友卡住（inactive 但任务没完成），让 Lead 唤醒它或改派。

第 3 步：验收（15 分钟）

逐条对照：

```
请按任务单的验收标准逐条检查三章，列一个表：文件 / 字数 / 有表格吗 / 有 3 个坑吗 / 术语解释了吗 / 通过与否。
不通过的直接返工，返工后重新验收。审查问题清单里的每条也要给结论（已修/不修+理由）。
```

然后**你自己也要读一遍**（`Ctrl+点击` 右侧文件树预览），确认不是"看起来完成"。

第 4 步：复盘（10 分钟）

写 `notes/day10.md`：

- 团队协作比单独干快/慢了？好在哪、差在哪；
- 实际花了多少 token（看用量）；
- 哪条任务书写得好、哪条让队友跑偏了；
- 下次会怎么拆。

完成标准

- ☐ 三章 + 审查清单 + README 全部产出
- ☐ 任务板上有 5 个任务，且有真实的依赖关系
- ☐ reviewer 挑出至少 3 条问题，且被处理（修了或说明不修）
- ☐ Lead 做过至少一次"唤醒/纠偏/改派"
- ☐ 你自己读过三章，确认质量
- ☐ 写完 `notes/day10.md` 复盘

常见坑

- ⚠️ **任务描述里没写"不许改别人的文件"**：队友会"顺手帮你优化"，冲突就来了。
- ⚠️ **验收标准写成"写得好"**：队友只能猜。**数字 + 清单。**
- ⚠️ **Lead 自己下场写**：Lead 的活是拆任务、协调、验收；写正文交给队友（除非只剩一小段）。
- ⚠️ **队友 inactive 就以为完成**：去任务板看状态、去文件看产出。
- ⚠️ **一个任务挂两个 owner**：任务板是"认领制"，一个任务一个负责人，需要多人就拆任务。
- ⚠️ **不停追问进度**：每次追问都在消耗 token 和打断节奏。**2–3 分钟一次足够。**

📋 今日提示词模板（流水线项目）：

```
工作目录：[路径]。
用 Agent Team 完成 [项目名]，产出到 [目录]。

任务板：
- 任务 1：[内容] / owner [A] / 写入范围 [文件] / 验收 [标准]
- 任务 2：[内容] / owner [B] / 依赖 任务 1 / 写入范围 [文件] / 验收 [标准]
- 任务 3：审查 / owner [C] / 依赖 任务 1、任务 2 / 产出 [问题清单]
- 任务 4：汇总合成 / owner Lead / 依赖 任务 3

规则：
1. 每个 owner 只写自己的文件，禁止交叉修改；
2. 每个任务消息自包含（队友看不到我们的对话）；
3. 你负责唤醒 inactive 队友、处理卡住的任务、按标准验收；
4. 完成后汇报：任务板变更次数、问题清单、最终文件。
```

第 11 天：Workflow——用脚本指挥一群 AI

今日目标

理解 Workflow 和 Subagent / Agent Team 的分工；让 DSH 写一个 workflow，对多个文件做批量审查。

核心概念

💡 **Workflow = 一段 JavaScript 编排脚本。**你在对话里描述"要对 N 个对象做同一套处理"，DSH 写脚本并运行：脚本里可以并行、可以分阶段、可以让每个子代理返回**结构化结果**（schema 校验），最后汇总。

💡 **和 Agent Team 的区别**：

	Agent Team	Workflow
谁决定下一步	Lead 和队友临场判断	**脚本预先写死**
适合	需要商量、返工、分工协作	**批量、重复、可流水线**的活
结果	文件 + 对话	**结构化 JSON**，可继续加工
可预测性	低（灵活）	高（稳定）

💡 **常用能力**：`agent(prompt)` 起一个子代理；`parallel([...])` 并发；`pipeline(items, stage1, stage2)` 流水线；`phase()` 分阶段进度；`log()` 打日志；`args` 传参；返回可序列化的结果。

💡 **什么时候用**：审计 30 个文件、批量改写标题、对 10 个网页做多角度研究、迁移代码。**重复 N 次、每次差不多**的活，用 workflow 比开团队便宜且稳定。

动手步骤

第 1 步：先跑一个小的（20 分钟）

在 `E:\AI-Learning\DSH练习` 新会话，粘贴：

```
工作目录 E:\AI-Learning\DSH练习。

请用 workflow 工具，对 notes/ 和 output/ 下所有 .md 文件做一次批量审查：
- 每个文件一个子代理，检查四件事：标题层级是否从 # 开始且不跳级、是否有连续空行、是否有明显错别字、字数；
- 并行处理，每个子代理只返回 JSON：{file, issues:[...], words}；
- 最后汇总成 output/audit-report.md：一张总表 + 按严重程度排序的问题清单。
- 运行前先把脚本给我看（打印出来），我确认后再跑。
```

它会把 workflow 脚本打印出来。**读懂三个部分**：

1. `meta`（名字、描述、阶段）；
2. 主体：怎么拿到文件列表、怎么 `parallel`、每个 agent 的 prompt 和 schema；
3. 返回值：汇总成什么。

第 2 步：加一个阶段（15 分钟）

> 改一下这个 workflow：加第二阶段——对第一阶段发现"有严重问题"的文件，
> 再起一个子代理给修改建议；最后报告里要有"建议"一列。
> 只改 workflow，不要手动改文件。

观察运行卡片：阶段、每个成员的进度、成功/失败数量。

第 3 步：对比成本（10 分钟）

同一批文件，用"主对话逐个读"做一遍，比较：

- 时间和 token；
- 你被打断的次数；
- 结果的一致性（两个文件的标准一样吗？）。

第 4 步：理解边界（5 分钟）

问 DSH：

> 用三句话告诉我：什么时候该用 workflow，什么时候该用 Agent Team，什么时候什么都不用、你自己做就行。

把它的回答和你自己的理解写进 `notes/day11.md`。

完成标准

- ☐ 成功运行一个 workflow，产出一份批量审查报告
- ☐ 能说出 workflow 脚本的三个组成部分
- ☐ 知道 `parallel` 和 `pipeline` 的区别（并发 vs 流水线）
- ☐ 给 workflow 加过一个阶段，并看到结构化结果
- ☐ 能说清 Workflow / Agent Team / 自己动手 三者的选择标准

常见坑

- ⚠️ **让 workflow 干"需要商量"的活**：它不会临场变通，剧本写完就按剧本走。
- ⚠️ **文件太多一次跑**：几十个并发子代理会烧钱且容易超时。**先 3–5 个试跑。**
- ⚠️ **不要求 schema**：结果变成一堆散文，没法汇总。**要求 JSON + 固定字段。**
- ⚠️ **忘了审查脚本**：脚本是代码，跑之前让它打印出来看一眼（有没有删文件、改原文件）。
- ⚠️ **把 workflow 当聊天**：它不回你消息，只返回结果；要交互就用 Team 或直接对话。

📋 今日提示词模板（批量处理）：

> 工作目录：[路径]。
> 用 workflow 对 [对象集合] 做 [同一套处理]：
> - 每个对象一个子代理，prompt 自包含：[检查/生成什么]；
> - 返回 JSON，字段固定：[字段列表]；
> - 并行/分阶段：[怎么编排]；
> - 最后汇总成 [文件路径]：[表格/排序/分级]。
> 运行前把脚本打印给我看，我确认后再跑。先只跑 [N] 个样本。

第 12 天：插件进阶——市场、实验功能、写一个自己的技能

今日目标

把插件页真正用起来：分清"装了什么"和"这个会话能用什么"；认识实验功能的代价；亲手写一个技能，让 DSH 以后按你的规程干活。

核心概念

💡 **插件页管"这台机器装了什么"，预设管"这个会话能用什么"。**DSH 把大部分工具从全局层"搬进了"每个会话的 agent 预设（standard / ptc / minimal）。所以会出现：插件页看不到 `tool-web` 的开关，但会话里 `web_search` 能用；或者某行显示已启用、会话里却没这个工具——**它不在当前预设里。**

💡 **插件页能做的六件事**：看组合包和每一行 → 开关组合包 → 开关某一行 → **添加插件**（包名 / Git / 压缩包 / 本地路径）→ 配（官方配置页：子智能体、网页搜索、Agent 循环、终端）→ 卸载。**实验性插件**单独一组，带"实验性"标签。

💡 **插件市场（Discover）本机默认没开**：它由 `dsh-experimental-plugin-market-bundle` 提供，不在本机启用的 bundle 里。想用要先在插件页的实验性分组里启用它，再"添加索引源"。注意：市场索引**没有签名、不支持一键升级**、只能装带 bundle 的包。

💡 **实验功能有代价**：

- **自动审查（Auto review）**：每次工具调用**多一次模型请求**，会明显变慢变贵，文档自己说可能误判——**先别开**，用"工作区内修改"权限更实在。
- **会话检查器（Inspector）**：会**未脱敏**采集网络请求，是排障工具，不是日常功能。
- **语音输入**：本地模型，首次要下载。
- **会话搜索 / Ralph / Worktree / 思维链翻译 / 持久终端工具**：本机都没启用，先知道有这回事。

💡 **技能（Skill）长什么样**：目录 `.dsh/skills/<名字>/SKILL.md`（或平铺 `<名字>.md`），开头是 YAML frontmatter：

```
---
name: weekly-report
description: 当用户要写周报、学习总结或阶段复盘时使用。
---
（正文：这类活该怎么干——步骤、模板、检查清单）
```

规则：`name` 必须小写连字符；`description` 是模型判断"什么时候用"的唯一依据（目录里只显示名字和这句描述）；**只扫一层目录**，不支持嵌套；正文没有大小上限，但只在加载时进入上下文。写完**立刻生效**，不用重启。

💡 **定时提醒 / 后台任务**：定时提醒（Schedule）是"到点给原会话发一条消息"，适合"每天 21:00 提醒我复盘"；后台任务是"长命令丢后台跑，不用等"，两者都在会话头部/侧栏能看到、能停。

💡 **MCP**：把外部工具服务器（GitHub、文件系统、数据库……）接成 DSH 的工具。它是普通插件 + 配置，需要手写 `cordis.patch.yml` 的一行。**知道概念即可，第一次配置建议等你能看懂配置文件。**

动手步骤

第 1 步：把插件页逛明白（15 分钟）

🖱️ 侧栏 → **插件**：

1. 在"已安装插件"里找到你认识的组合包（如 `dsh-web-app`），点进去看它包含哪些行；
2. 找到"实验性插件"分组，看有哪些带**实验性**标签；
3. 找到"内置插件配置"分组里的官方配置页：**子智能体、网页搜索、Agent 循环、终端**——各点开看一眼有哪些可调项（**先别改**）；
4. 记下三个发现，写进 `notes/day12.md`。

> 注意：这里**没有"发现"标签页**是正常的——本机的插件市场 bundle 没启用。

第 2 步：开关一次实验插件（10 分钟）

🖱️ 在实验性分组里找到"语音输入"，把它关掉（或打开），观察提示（可能需要重开会话/应用才生效）。确认没有影响后再改回来。

> ⚠️ 只关你认识的；不认识的不要动。改坏了怎么办：见附录 F。

第 3 步：写你的第一个技能（25 分钟，今天核心）

先建目录，再让 DSH 帮你写：

```
工作目录 E:\AI-Learning\DSH 练习。

请帮我创建一个技能，放在 .dsh/skills/weekly-report/SKILL.md：
- 用途：当我要求"写周报 / 本周总结 / 学习复盘"时使用；
- 正文包含：写作步骤（先问我这周做了什么 → 按四块组织 → 自检）、
  固定结构（本周完成 / 卡点与解决 / 下周计划 / 一句话总结）、
  风格要求（中文、每条不超过两行、不要形容词堆砌）；
- frontmatter 的 name 用 weekly-report，description 用一句话说明什么时候用；
- 写完后告诉我这个技能会被怎么发现、怎么加载。
```

写完后**新开一个会话**（重要：技能目录在首次请求前注入），发：

```
用 weekly-report 技能，帮我写本周（第 1 周）的学习周报。这周我做了：
[写 3–5 条你真实做过的事]
```

如果它没有自动命中，试试直接输入 `/weekly-report`（用户显式调用），再观察。

第 4 步：定时提醒与后台任务（10 分钟）

> 1）帮我建一个每天 21:00 的提醒：内容是"花 5 分钟更新 docs/progress.md"。
> 2）再跑一个后台任务：用 PowerShell 等 30 秒然后输出"后台任务完成"，不要阻塞我们的对话。

🖱️ 去侧栏的**自动化任务**页面看第一个；在会话头部看第二个（可以展开实时输出、可以停止）。

第 5 步（可选）：打开插件市场（10 分钟）

想装第三方插件时再做：插件页 → 实验性分组 → 启用"插件市场" → 重启 DSH → 插件页出现"发现" → 添加索引源（一个市场 URL）→ 浏览、安装。

> ⚠️ 第三方插件是陌生代码，装之前想清楚；学习期不建议装。

完成标准

- ☐ 能说清"插件页的开关"和"会话里能用的工具"为什么不是一一对应
- ☐ 逛过实验性插件分组和官方配置页
- ☐ 写好了 `weekly-report` 技能，并在新会话里成功命中（或 `/weekly-report` 调用成功）
- ☐ 成功创建并查看了一个定时提醒
- ☐ 成功跑了一个后台任务并看到输出
- ☐ 知道插件市场默认没开、开它需要什么
- ☐ 能说出至少 3 个实验功能的代价

常见坑

- ⚠️ **在插件页乱关行**：可能让某个界面功能消失。**只动带"实验性"标签的 bundle**，出问题按附录 F 回退。
- ⚠️ **技能 `name` 写错格式**：必须小写连字符（`weekly-report`，不能 `WeeklyReport`），frontmatter 写错技能会**静默消失**。
- ⚠️ `description` **写成"这是什么"**：模型靠它判断"什么时候用"，要写成触发条件（"当用户要写周报时使用"）。
- ⚠️ **把技能目录放错层**：只扫 `.dsh/skills` 或 `.agents/skills` 下一层，不要写成 `.dsh/skills/a/b/SKILL.md`。
- ⚠️ **开了自动审查**：每次工具调用多一次模型请求，明显变慢变贵，还不一定准。
- ⚠️ **手改** `cordis.patch.yml` **不备份**：它是整块替换配置，写漏一个键就等于删掉它。**改前复制一份 .bak**。

📋 今日提示词模板（写技能）：

> 工作目录：[路径]。
> 帮我创建一个技能，放在 .dsh/skills/[name]/SKILL.md：
> - 用途（什么时候用）：[触发条件，写具体]
> - 正文：步骤 / 固定结构 / 风格要求 / 自检清单
> - frontmatter：name=[kebab-case]，description=[一句话触发条件]
> 写完告诉我：它会被怎么发现、怎么加载、怎么验证。

第 13 天：结营项目（上）——选题、计划、组队开工

今日目标

独立完成一次"项目启动"：定题、写规格、拆任务板、组建团队并开工。今天不追求做完，追求**做对**。

选题（三选一，或自定）

选项	产出	难度	说明
A. 个人作品集网站	一个多页静态站 + GitHub Pages 链接	⭐⭐⭐	复用第 7 天的主页，扩展成"作品集"：项目页、学习记录、关于我
B. 一本电子书	《DSH 上手指南》Word + PDF	⭐⭐⭐	用 Agent Team 分工写章节，用 Office 技能排版导出
C. 学习数据看板	一个 Excel/网页看板 + 数据说明	⭐⭐⭐	把 14 天的学习记录（时间、token、产出）做成可视化

> 推荐 **A**（最像"作品"，能发给别人看）；想练文档能力选 **B**；喜欢数据选 **C**。

动手步骤

第 1 步：写项目规格（20 分钟）

在 `E:\AI-Learning\DSH练习` 新开会话，用第 4 天的计划模式：

```
/plan

工作目录 E:\AI-Learning\DSH练习。
我要做结营项目：[选项 A/B/C]。

请先帮我把"项目规格"写清楚，输出到 docs/spec.md：
1. 一句话目标 + 给谁看；
2. 成品清单（每个文件是什么、放在哪）；
3. 验收标准（至少 5 条，可检查）；
4. 不做什么（范围外）；
5. 风险与依赖（哪些需要我先确认）。
先读 docs/ 下已有文件，再写；写完给我看，不要开始做项目。
```

你审一遍 `spec.md`，改到满意为止。**规格没定清，后面全是返工。**

第 2 步：让 Lead 拆任务板（15 分钟）

```
根据 docs/spec.md，用 Agent Team 拆任务板，先不要创建队友、不要开工：
- 把项目拆成 5–8 个可独立验收的任务；
- 每个任务写：目标、输入、产出路径、验收标准、写入范围、依赖关系；
- 标出哪些任务可以并行、哪些必须串行；
- 列出你打算创建的队友（名字 + 职责），并说明为什么这样分。
输出一张任务表到 docs/tasks-plan.md，等我确认。
```

你自己检查三件事：

1. 有没有两个任务写同一个文件？（有就合并或拆开）
2. 每个任务能不能独立验收？
3. 依赖关系对不对（先调研后写作、先写作后审查）？

第 3 步：组队开工（35 分钟）

确认后发：

```
按 docs/tasks-plan.md 开工：
1. 创建你计划里的队友（fresh 启动）；
2. 按依赖顺序给每个任务指派 owner，任务描述要自包含；
3. 你负责协调和验收，每完成一个任务向我汇报一次；
4. 遇到需要我决策的（改范围、加人、删文件、装东西），停下来问我；
5. 所有产出写进 docs/ 约定的路径，禁止交叉修改。
现在开始，先创建任务板，再创建队友，然后逐个派活。
```

第 4 步：当项目经理（20 分钟）

- 每 2–3 分钟看一眼团队面板和任务板，不要频繁追问；
- 有队友 inactive 但任务没完成：让 Lead 用 `send_message` 唤醒或改派；
- 发现方向跑偏：立刻纠偏（"第 2 章的读者是零基础，不要出现代码"）；
- 记录：开始时间、队友数量、token 用量（`notes/day13.md`）。

完成标准（今天只验收"启动"）

- ☐ `docs/spec.md` 写完，验收标准 ≥ 5 条且可检查
- ☐ `docs/tasks-plan.md` 有 5–8 个任务，依赖关系合理
- ☐ 没有两个任务写同一个文件
- ☐ 队友创建成功（≥ 2 人），任务板已派活
- ☐ 至少一个任务已"进行中/已完成"
- ☐ 你在 `notes/day13.md` 记下了启动时间和 token 用量

常见坑

- ⚠️ **规格没写就开工**：做到一半发现方向不对，全部返工。**今天最重要的产出是 spec。**
- ⚠️ **任务拆得太粗**（"做网站"）：队友无法独立完成，Lead 也无法验收。
- ⚠️ **任务拆得太细**（20 个任务）：协调成本超过干活成本。**5–8 个刚好。**
- ⚠️ **所有任务都串行**：团队优势没了。能并行的（三个独立章节/页面）一定要并行。
- ⚠️ **忘了写"不做什么"**：队友会热情地加需求。

📋 今日提示词模板（项目启动）：

```
/plan
工作目录：[路径]。
项目：[一句话]。
产出：[文件清单 + 路径]。
验收：[5 条以上可检查标准]。
先写 docs/spec.md，我确认后再拆任务板（5–8 个任务，标依赖与写入范围），
再创建 [N] 个队友开工。需要我决策的停下来问。
```

第 14 天：结营项目（下）——验收、发布、复盘

今日目标

把项目收尾、发布、复盘，并给自己定下一个 30 天计划。今天做完，你就有了第一份"AI 协作作品"。

动手步骤

第 1 步：继续并收尾项目（30 分钟）

```
继续 docs/tasks-plan.md 的剩余任务，要求：
1. 所有任务必须"完成"或明确说明"不做了+原因"；
2. 做一次整体验收：按 docs/spec.md 的验收标准逐条检查，列一张"标准 / 结果 / 证据"的表；
3. 检查一致性：术语、编号、链接、文件路径；
4. 把最终清单写到 docs/progress.md。
```

然后**你自己逐条验收**（不要只看它的表）：

- 文件都在吗？能打开吗？
- 链接能点吗？手机上看过吗？
- 有没有明显编造的内容？

第 2 步：发布（20 分钟）

选 A 的：推到 GitHub 并开 Pages（同第 7 天流程），把链接写进 README。

选 B 的：用 Office 技能导出最终 PDF，在 GitHub 建 Release 附上 PDF 和 Word。

选 C 的：把 Excel/网页和数据一起提交，README 写清怎么打开。

```
cd E:\AI-Learning\DSH练习
git add -A
git commit -m "feat: 完成结营项目"
git push
```

第 3 步：导出与存档（10 分钟）

- 在最重要的几个会话里执行 `/export`，下载会话日志（ZIP）存档；
- 把 `docs/`、`output/`、`notes/` 的关键文件列一份清单到 `docs/handoff.md`；
- 确认没有把隐私/密钥提交进仓库（`git log` + 搜一遍敏感词）。

第 4 步：算一笔账（10 分钟）

设置 → 账号 / 通用设置 → 性能与用量，记下：

指标	数值	说明
总 token / 缓存命中率		命中越高越省钱

指标	数值	说明
会话数 / 最长会话		是不是该早点分段
团队协作 vs 单人做的花费		团队值不值
高峰时段用量占比		错峰能省一半

写进 `notes/day14.md`。

第 5 步：复盘（20 分钟）

> 帮我写一份 14 天复盘，保存到 docs/retro.md，结构：
> 1. 我学会了什么（按"会用 / 提示词 / 团队 / 插件"四块，每块 3 条）；
> 2. 最卡的 3 个地方 + 怎么解决的；
> 3. 如果重来一次，我会改变哪 3 个做法；
> 4. 我最想继续深入的 1 个方向。
> 要求：用第一人称，内容基于我们这 14 天的真实记录（读 notes/ 和 docs/），不要编造。

然后**自己改一遍**——复盘是给你自己看的，不是给 AI 看的。

第 6 步：终极自测（15 题，至少答对 12 题）

1. 工作区、会话、文件三层分别存什么？
2. Agent 的工具循环是哪四步？为什么"它看不见就不能用"？
3. 三档权限分别允许什么？本机默认哪档？为什么建议改？
4. 上下文为什么会越用越贵？压缩能救什么、救不了什么？
5. 提示词六要素是什么？各缺一个会怎样？
6. `@` 引用到底做了什么？为什么它不等于"把文件内容发给模型"？
7. 计划模式的"批准"是安全护栏吗？
8. AGENTS.md 的加载顺序？改了以后旧版本会消失吗？
9. 什么时候写 AGENTS.md，什么时候写技能？
10. 插件页的开关和会话里的工具为什么不一致？
11. 子代理和 Agent Team 的四个区别？
12. Agent Team 里 Lead 独占哪三个权限？写入范围能阻止冲突吗？
13. 队友 inactive 代表什么？Lead 这时该做什么？
14. Workflow 和 Agent Team 怎么选？
15. 说出 5 个自带功能，并说明各自解决什么问题。

第 7 步：定下一个 30 天计划（10 分钟）

三条路线，选一条写进 `docs/next-30-days.md`：

1. **继续深挖 DSH**：学插件开发（Cordis 插件）、MCP 接入、Workflow 复杂编排；
2. **用 DSH 学编程**：HTML/CSS/JS 或 Python，用 DSH 当老师 + 结对程序员，做 2 个小项目；
3. **用 DSH 做内容**：把 14 天所学写成教程/视频脚本，发布到 GitHub + 平台，建立作品集。

> 三条路都指向同一件事：**持续产出**。工具会过时，"把事做完"的能力不会。

完成标准

- ☐ 项目所有任务完成或明确取消，并写下原因
- ☐ 按验收标准逐条验收，有"标准 / 结果 / 证据"表
- ☐ 作品已发布（Pages 链接 / Release / 仓库）
- ☐ 导出并归档了关键会话
- ☐ 算清了 token 与花费，知道哪里可以省
- ☐ `docs/retro.md` 写完并自己改过
- ☐ 终极自测答对 ≥ 12 题
- ☐ `docs/next-30-days.md` 定好方向

常见坑

- ⚠️ **验收只看它的汇报**：一定要自己打开文件看。它说"已完成"不等于完成。
- ⚠️ **发布前不检查隐私**：仓库一旦公开，历史里删不干净。
- ⚠️ **不导出会话**：换电脑/重装就只剩作品，没有过程记录。**过程也是作品的一部分。**
- ⚠️ **复盘写成流水账**：写"下次怎么做"才有用，写"我学了什么"只是给自己看。
- ⚠️ **做完就停**：14 天的价值在于形成习惯。**下一个 30 天，哪怕每天 30 分钟，也要持续产出。**

📋 今日提示词模板（验收）：

```
按 docs/spec.md 的验收标准逐条验收，输出表：标准 / 结果 / 证据（文件+行号或截图说明）。
不合格的列出问题并修复；修完重新验收。
最后给我：最终文件清单、已知问题、下一步建议。
```

附录

附录 A：提示词手册

A1. 一条提示词的骨架

> [角色]（可选）
> 目标：……
> 背景：……
> 输入：……（路径/文件/链接）
> 约束：……（不许做什么、风格、长度、语言）
> 验收：……（可检查的标准）
> 输出：……（写到哪、什么格式）

六格缺一格会怎样（这是新手最容易踩的坑）：

缺了什么	模型会怎样	典型后果
目标	猜一个它以为的目标	做得很努力，但不是你要的
背景	用通用假设	内容空泛、对象错位
输入	凭训练数据编	数据过时或编造
约束	自由发挥	太长、跑题、动了不该动的文件
验收	不知道何时停	提前收工或过度加工
输出格式	只在对话里给你	关掉会话就找不到成品

A2. 十条可操作原则

1. **先写目标与验收**：模型没有"完成"信号，验收必须由你说。
2. **给路径，不贴长文**：长文会作为历史反复计费；文件让模型自己读。
3. **一次只给一件可验收的事**：五件事混一轮，验收点互相稀释。
4. **写清边界**："只改 X，不要动 Y，需要越界先问我"。
5. **规定输出格式**：三段式（结论 / 证据 / 风险）比"分析一下"有用得多。
6. **先计划后执行**：大改动先 `/plan`，拿一个人类批准点。
7. **不确定就让它先问**：把"拿不准就问，给推荐项"写进提示词。
8. **返工说清"差在哪、改成什么"**：只指出问题、划范围、要验收结果。
9. **上下文卫生**：一个可验收的子任务一个会话；换阶段先写交接文件。
10. **用文件当记忆**：结论写进 `docs/`、`notes/`，不要指望对话记住。

A3. 十二个可复制模板

1）新建任务

> 工作目录：[绝对路径]。
> 目标：[一句话最终结果]。
> 背景：[给谁/为什么/什么基础]。
> 输入：[读哪些文件，不读什么]。
> 约束：只改 [范围]；不要动 [禁止清单]；[风格/长度/语言]。
> 验收：[可检查标准 1]；[标准 2]。
> 输出：[文件路径 + 格式]。
> 不确定时先问我（给推荐项和理由），不要猜。

2）返工

> 上一版的问题：[具体到文件/段落，1–3 条]。
> 期望改成：[具体形态，最好给对照例子]。
> 保持不变：[不要顺手改的部分]。
> 只改这些；改完逐条对照验收标准复述结果。

3）调研

> 问题：[一句话]。
> 范围：只查 [路径/文档/URL 列表]。
> 方法：先列证据，再给结论；每条结论后标 [文件:行] 或 URL。
> 不要：凭印象补细节；找不到就写"未找到"。
> 输出：结论 ≤3 条 + 证据表 + 仍不确定的点。

4）写文档

> 读者：[谁看、什么基础]。
> 目标：读完能 [做到什么]。
> 结构：[小节标题列表]。
> 素材：只用手头的 [文件/链接]，不要编造。
> 风格：中文，术语保留英文并解释，代码块要能直接跑。
> 写到 [路径]；不要改其他文件；写完列出引用出处。

5）审查代码/文档

> 审查对象：@[文件或目录]。
> 关注：[正确性/一致性/可执行性/与文档是否相符]。
> 产出：按严重度排序的问题清单，每条 = 位置 + 问题 + 建议改法。
> 先给清单，我挑完你再改；没问题的部分明确写"未发现问题"，不要凑数。

6）交接换会话

> 先读 [notes/progress.md 或 docs/handoff.md]，那是上一段的结论。
> 当前目标：[一句话]。
> 已完成：[≤3 条]。
> 下一步：[一件事]。

> 约束：[已定死、不要再讨论的决定]。
> 先复述你理解的状态，再开工。

7）先复述计划

> 动手前用 5 行以内回答：
> 1）你要改哪些文件 2）每处改什么 3）怎么验证 4）有哪些不确定。
> 我确认之前不要写文件、不要跑会改状态的命令。

8）计划模式

```
/plan

工作目录：[路径]。
目标：[结果]。
现状：[已知文件/结构/限制]。
要求：[步骤约束，如"不删文件""只新建不改旧"]。
先探索再给计划：分步写清做什么、动哪些文件、怎么验证。
不要动手，等我批准；我驳回时按意见修改后重新提交。
```

9）并行调研（子代理）

```
工作目录：[路径]。
用 [N] 个子代理并行完成以下调研，全部回来后再汇总：
- 任务 A：[目标/读什么/产出/格式]
- 任务 B：……
每个子代理写 output/research/[名称].md；
汇总成 [报告路径]，标注每节来源；不许编造。
```

10）组建团队（Agent Team)

```
工作目录：[路径]。
用 Agent Team 完成 [目标]，产出 [文件]。

任务板：
- 任务 1：[内容] / owner [A] / 写入范围 [文件] / 验收 [标准]
- 任务 2：[内容] / owner [B] / 依赖 任务 1 / 写入范围 [文件] / 验收 [标准]
- 任务 3：审查 / owner [C] / 依赖 任务 1、2 / 产出 [问题清单]
- 任务 4：汇总 / owner Lead / 依赖 任务 3

规则：每个 owner 只写自己的文件；任务描述自包含（队友看不到我们的对话）；
你负责唤醒 inactive 队友、处理卡住的任务、按标准验收；
完成后汇报：任务板变更次数、问题清单、最终文件。
```

11）批量处理（Workflow)

```
工作目录：[路径]。
用 workflow 对 [对象集合] 做 [同一套处理]：
- 每个对象一个子代理，prompt 自包含；
- 返回 JSON，字段固定：[字段]；
- 并行/分阶段：[编排方式]；
```

> - 汇总成 [报告路径]。
> 运行前把脚本打印给我看；先只跑 [N] 个样本。

12）定时/后台

> 帮我建一个 [每天 21:00 / 每周日 10:00 / 明天 9:00] 的提醒：
> 内容是"[提醒文本]"，我在这个会话里等它。
> 另外把 [长命令] 丢到后台跑，不要阻塞对话；跑完把结果摘要给我。

A4. 好提示词 vs 坏提示词（对照）

坏	好	为什么
帮我整理一下文件	把桌面 `待整理/` 里的文件按"图片/文档/其他"分到三个子目录，生成一份清单到 `notes/整理清单.md`，不要删除任何文件	有目标、范围、产出、约束
写个总结	读 @docs/进度.md，写 200 字总结给没参与项目的人看，三段：做了什么/现在在哪/下一步	有读者、输入、结构、长度
优化一下代码	只改 `src/a.ts` 的 `parse()` 函数，让它处理空输入不报错；加一个测试；不要动其他函数	有范围、验收、边界
你觉得怎么办	给我 3 个方案，每个写清成本/风险/推荐理由，最后给推荐项	强制结构化输出
重做	第 2 段数据错了：2026 年不是 2025 年。只改这一处，其余保持	返工要具体

A5. 反模式清单

- ❌ 把"要专业/要好看/要口语化"当要求 → ✅ 换成可数的标准（字数、段数、术语表、示例数）
- ❌ 一次塞五个任务 → ✅ 拆成五条，逐条验收
- ❌ 贴 300 行日志 → ✅ 给路径 + 让它切片读（"先读前 100 行，再 grep 关键字"）
- ❌ 不提验收 → ✅ 至少 2 条可检查标准
- ❌ 让它在对话里交付 → ✅ 要求写入文件
- ❌ 出问题就重开一轮 → ✅ 用返工句式在原会话修
- ❌ 以为改 AGENTS.md 会"替换"旧规则 → ✅ 它追加新消息，旧文本仍在历史里
- ❌ 以为 `@文件` 就把文件内容发给模型了 → ✅ 它只插入路径，模型要自己读
- ❌ 以为计划模式能阻止它动手 → ✅ 它只是文本约束，真正的护栏是权限 + 沙箱

- ❌ 以为问题超时没回答=默认同意 → ✅ 超时不等于批准

附录 B：Agent Team 手册

B1. 一页看懂

概念	大白话	关键事实
Lead	你正在聊的这个会话，就是"队长"	隐式存在，不需要创建；`TeamId` = 会话 ID
teammate（队友）	有名字、有职责、有独立持久会话的 AI	只能由 Lead 创建；名字永久、不复用
共享任务板	团队的白板，谁干什么、卡在谁那	任务有 owner、依赖、写入范围、revision
inbox / 消息	队友的收件箱	消息会进目标的历史（带发送者名字前缀）
写入范围 write_scopes	"这个任务大概会改哪些文件"	**只是提示**：重叠只警告，不阻止
依赖 blocked_by	"任务 A 没完成，任务 B 不能开始"	依赖全部完成，任务才"可开始"
revision	任务的版本号	每次变更都带版本，防止互相覆盖（旧版本操作会被拒）

B2. 工具清单

工具	谁可以用	作用	关键点
`spawn_teammate`	**仅 Lead**	创建一个具名队友	参数：`name`（小写连字符）、`description`、`prompt`（完整初始任务）、`context`（`fresh` 无记忆 / `fork` 继承已完成轮次）
`send_message`	所有成员	给另一个成员发消息	running 的目标在**最近的步骤边界**收到；inactive 的目标会被**唤醒**；返回"已入箱"，不代表已处理
`list_agents`	所有成员	看成员名册与状态	无参数；只反映"当前可寻址的成员与可用性"

工具	谁可以用	作用	关键点
`wait_agent`	所有成员	等下一次团队变化	只观察**调用之后**的状态/message/任务变化；**不会唤醒**任何人；没有活跃成员时立即返回 noProgress；超时 10 秒–1 小时，默认 30 秒
`interrupt_agent`	**仅 Lead**	中断队友当前轮次	**保留**它排队的消息；不释放任务 owner
`team_task_create`	所有成员	创建待办任务	参数：`subject`、`description`、`blocked_by`、`write_scopes`
`team_task_list`	所有成员	列任务	可按 `status`/`owner`/`ready` 过滤；`owner: "unowned"` 查无主任务
`team_task_get`	所有成员	读单个任务（含最新 revision）	改任务前必须先读
`team_task_update`	所有成员（部分操作仅 Lead）	改任务	必须带 `expected_revision`；`action`：claim / release / edit / set_dependencies / complete / reopen / **reassign（仅 Lead）** / delete

> 本机配置：最多 **8** 个队友（默认上限 16）、**256** 个活动任务、单条消息 **64KB**。

B3. 成员状态

状态	含义	你该做什么
`provisioning`	正在创建	等（它可能还在启动）
`running`	正在执行一轮	别打扰；等结果或 `wait_agent`
`inactive`	**当前没有执行轮次**（可能在等消息，也可能干完了，也可能只是没被叫醒）	**先看任务板和产出**；需要它继续就 `send_message`
`failed`	创建失败	名字已占用、不能复用；换名字重建或改派任务

> 记住一句话：**inactive ≠ 完成**。完成与否看任务板 + 文件。

B4. 标准协作流程

1. **Lead 拆任务**：5–8 个，每个可独立验收；写清目标/输入/产出/验收/写入范围/依赖。
2. **建任务板**：`team_task_create`，有先后关系的用 `blocked_by`。
3. **创建队友**：`spawn_teammate`，prompt **自包含**（fresh 队友看不到你的对话）。
4. **派活**：任务指给 owner（`team_task_update` reassign），或让队友自己 claim。
5. **盯进度**：`list_agents` + `team_task_list`；队友 inactive 且任务没完成 → `send_message` 唤醒；有活跃队友时才 `wait_agent`。
6. **验收**：读产出，按验收标准逐条核对；不合格 → 返工（改任务/发消息）。
7. **收尾**：任务 complete；`docs/progress.md` 更新；Lead 给最终答案前必须等齐必需的队友。

B5. 已知限制（重要，必须知道）

限制	影响	应对
单进程、共享同一目录	成员改的文件立刻互相可见；没有隔离、没有文件锁、没有自动合并	**按文件/目录分工**；Lead 最后检查 diff
写入范围只是提示	重叠只会警告，不会阻止覆盖	任务设计上避免重叠
扁平不可嵌套	只有 Lead 能建人；队友不能再建团队	任务分片，不要幻想"分队长"
名字不可复用/不可删	失败的名字也永久占用	起名想清楚；失败就用新名字
owner 不会自动释放	队友挂了，任务还挂它名下	Lead 用 reassign/claim 处理
没有发送重试/去重保证	进程崩了，未送达的消息不会补发	关键信息落文件，不靠消息
任务 CAS	基于旧 revision 的修改会被拒	改前先 `team_task_get` 拿最新 revision
开销放大	每个队友都是独立上下文和独立计费	小任务别开团；2–3 人起步

B6. 什么时候用 / 不用

适合

- 一个任务可以拆成 3–8 个相对独立的片（三个章节、三个页面、三个模块）；
- 需要"分工 + 串行依赖 + 审查"的流水线（调研 → 写作 → 审查）；
- 需要多个视角互相挑错（红队审查）；
- 任务跨多轮，队友的状态和产物需要保留、可恢复。

不适合

- 小而简单、一个人（单会话）20 分钟能做完的事；

- 必须频繁你来我往拍板的探索型任务（每次都要你决策，团队协调成本更高）；
- 多个成员必须改同一个文件（冲突几乎必然）；
- 你无法验收的任务（团队产出多，你判断不了对错就更危险）。

B7. 一个完整示例

```
工作目录 E:\AI-Learning\DSH 练习。
用 Agent Team 做一个「DSH 学习指南」小册子（3 章 + 1 次审查），产出到 output/guide/。

1. 创建 4 个队友：writer-a、writer-b、writer-c、reviewer。
2. 任务板：
   - T1 第 1 章「认识 DSH」，owner writer-a，写入范围 output/guide/ch1.md；
     验收：800-1200 字、含 1 张表、含 3 个新手坑。
   - T2 第 2 章「提示词」，owner writer-b，写入范围 output/guide/ch2.md，同上。
   - T3 第 3 章「Agent Team」，owner writer-c，写入范围 output/guide/ch3.md，同上。
   - T4 审查三章，owner reviewer，blocked_by T1,T2,T3，写入范围 output/guide/review.md；
     验收：每章至少 2 条具体问题，不许直接改别人的文件。
   - T5 汇总，owner Lead，blocked_by T4，产出 output/guide/README.md（目录+摘要）。
3. 每个任务描述要自包含：素材只允许来自 E:\AI-Learning 下现有文档，不许编造。
4. 任一队友 inactive 且任务未完成，先 send_message 唤醒；没有活跃成员时不要空等。
5. 验收通过后向我汇报：任务板变更次数、reviewer 挑出几条、最终文件清单。
限制：两人不许写同一个文件；超过 8 个任务或出现重叠先停下来。
```

B8. 新手常见误解（10 条）

1. "建了团队 AI 就会自动分工"——分工是 **Lead 的任务书**决定的，任务写得烂，团队就乱。
2. "队友是子代理的升级版"——队友是**持久**的、能互相通信的；子代理是一次性的。两者用途不同。
3. "inactive 就是完成了"——inactive 只表示"现在没在跑"。
4. "写入范围能防止冲突"——它只是提示，冲突要靠分工避免。
5. "队友能看到我们的对话"——默认 `fresh` 看不到，任务必须自包含；`fork` 才继承。
6. "Lead 可以下场写正文"——可以，但 Lead 的核心价值是拆解、协调、验收。
7. "任务改一下随便改"——任务板是版本化的，旧 revision 的修改会被拒绝。
8. "团队越大越好"——每个成员都在独立烧 token；2–3 人通常是性价比最高的。
9. "队友失败可以重名重建"——名字永久占用，失败也要换新名字。
10. "等不到就反复轮询"——`wait_agent` 观察变化、`list_agents` 看状态、`send_message` 唤醒，顺序别反。

附录 C：自带插件目录

C1. 先理解三件事

1. **"包多"不等于"插件多"**：本机安装包里约有 306 个 npm 包，但你在界面上能感知的功能大约 40 项。绝大多数包是内部实现细节（文件系统、会话格式、沙箱、存储……），普通用户不需要认识。
2. **插件页管"这台机器装了什么"，预设管"这个会话能用什么"**：DSH 把大部分工具从全局层"搬进"了每个会话的 agent 预设（`standard` 默认 / `ptc` / `minimal`）。所以插件页看不到 `tool-web` 的开关，但会话里 `web_search` 照样能用。
3. **一句话口诀**：看得见的东西 → **插件页**；看不见但会用的工具 → **预设**；外部世界 → **MCP**；一次性的说明书 → **技能（skill）**。

C2. 你能直接感知的自带功能

① 对话与界面

功能	包名（了解即可）	作用	在哪看到	本机
聊天与会话流	`dsh-client-ui-chat`	主界面；工具调用折叠成组，收尾挂动作栏	主区域	开
会话列表与搜索	`dsh-client-ui-sidebar`、`ui-workspace`	按工作区分组、置顶、重命名、fork、归档	左侧栏；`Mod+K` 搜索	开
模型与推理等级	`dsh-client-ui-model-selection`	给当前会话换模型/推理档	输入框模型入口、`/model`	开
权限模式	`dsh-client-ui-permission-presets`	仅可查看 / 工作区内修改 / 完全权限（+实验的自动审查）	输入框权限控件、`/permission`	开；默认**完全权限**
计划模式	`dsh-plan-mode`、`ui-plan`	只准调研、出计划给你批准	输入框计划入口、`/plan`	开（`minimal` 无）
目标	`dsh-goal`、`ui-goal`	给会话挂长期目标，跨轮次推进	输入框上方 GoalBar、`/goal`	开（`minimal` 无）
`@` 引用	`dsh-client-ui-reference`	`@` 选文件/文件夹/历史会话	输入框敲 `@`；`@"带 空格的路径"`	开
`/` 命令	`dsh-client-ui-commands`	命令菜单，避免把 `/xxx` 当普通提问发出去	输入框敲 `/`	开

② 工作区与文件

功能	包名	作用	在哪看到	本机
工作区选择	`ui-workspace`、`host-directory-picker-auto`	给会话挑目录；注册工作区	侧栏"添加工作区"、新会话主视觉	开
右侧栏文件树	`dsh-client-ui-sidebar-files`	浏览工作区、点文件预览	右侧栏"新标签页 → 文件"；`Mod+P`	开
文档预览	`dsh-client-ui-sidebar-documentpreview`	Markdown/代码/图片/PDF/HTML/Excel 直接看；Word/PPT 本地转 PDF 后预览	文件树、改动卡片、`@` 引用	开
用户终端	`dsh-client-ui-sidebar-terminal`	在会话目录开真正的交互 shell，刷新后还在	右侧栏"新建终端"	开
浏览器标签	`dsh-client-ui-sidebar-browser`	右侧栏内嵌网页（本机 loopback 也行）	右侧栏"浏览器"；`Ctrl+T`	开（desktop）
打开方式	`dsh-client-ui-open-in-app`	用本机应用打开目录/文件	会话头部、文件树标题栏	开
交付/改动卡片	`ui-deliverables`、`dsh-workspace-changes`	收尾显示"交付的文件"和"本轮改了哪些文件+行数"，点开看 diff	每轮回答末尾	开

③ 能力类（模型的手脚）

功能	包名	作用	备注
文件读写改	`tool-fs`、`fs-sandbox`、`fs-observation-policy`	`read`/`write`/`edit`；读后才能改，越界被沙箱拦	每个会话都挂
PowerShell / bash	`tool-pwsh`、`tool-bash`	跑命令、装依赖、跑测试；长命令丢后台	Windows 上只有 PowerShell 系
文件搜索	`tool-fs-search`	`glob` 找文件名、`grep` 找内容	内置查找
后台任务	`jobs-local`、`tool-jobs`、`ui-jobs`	长任务后台跑、看实时输出、停止	会话头部
定时提醒	`dsh-schedule`、`tool-schedule`、`ui-schedule`	到点给原会话发消息；持久化、重启还在	侧栏"自动化任务"

功能	包名	作用	备注
网页搜索/抓取	`tool-web`、`web-deepseek-api`	`web_search` / `web_fetch`，结果是**外部不可信数据**	本机走新版 Search/Fetch API
Office 文档	`dsh-skill-office`	Word/Excel/PPT 生成 + 校验 + 渲染 + 导出 PDF	通过技能触发
语音输入	`experimental-voice-input-bundle`	按住麦克风说话转文字	首次需下载本地模型

④ 协作与扩展

功能	包名	作用	本机
子代理	`dsh-subagent`、`tool-subagent`	独立上下文的子代理；fork 版继承对话	开（standard 预设）
Agent Team	`experimental-agent-team` 系列	Lead + 具名队友 + 共享任务板 + 互发消息	开（bundle 已启用）
Workflow	`workflow-ptc`、`tool-workflow`、`ui-workflow-run`	写 JS 脚本给大量子代理分派批量任务	开（standard）
技能系统	`dsh-skill`、`skill-filesystem`、`tool-skill`	按需加载的"操作规程"	开
MCP	`mcp-resources`（客户端 `mcp-client` 未挂）	接外部 MCP 服务器的工具/资源	需配置
插件管理器	`client-ui-plugin-manager`、`plugin-manager`	侧栏插件页：开关、安装、卸载	开
插件市场	`client-ui-plugin-market`	浏览第三方市场并安装	**bundle 未启用**（无"发现"页）

⑤ 实验性功能（本机已装 bundle，但请谨慎）

功能	作用	代价	本机
自动审查 Auto review	每次工具调用前用模型判一次风险（low 放行 / medium 审批 / high 拒绝）	每次调用**多一次模型请求**；会误判	已装，默认未选
会话检查器 Inspector	看会话原始事件流 + Node 诊断	**未脱敏**采集网络请求；排障用	已装（`Ctrl/Cmd+Shift+.`）
语音输入	本地 SenseVoice 转写	首次下载模型	已装
会话搜索	给模型 5 个搜历史会话的只读工具	侧栏搜索默认只搜标题/工作区名	未启用

功能	作用	代价	本机
Ralph	"写-跑-修"自动循环（最多 64 轮）	烧 token	未启用
Worktree	给模型开 git worktree 的工具	需要 git 概念	未启用
思维链翻译	把思考过程翻译成目标语言	额外请求	未启用
会话标题（全提示词版）	用全部提示词生成标题	顶掉默认标题策略	未启用
持久终端工具 / 徽章技能	给模型 6 个持久 shell 工具；给产出加"powered by dsh"徽章	小众	未启用

C3. 插件页怎么用（六件事）

动作	怎么做	注意
看	侧栏 **插件** → 已安装/实验性分组	点进组合包能看到它的每一行
开关组合包	卡片上的开关	写 `package.json` 的 bundle 列表；关闭不删依赖
开关某一行	组合包详情里逐行开关	只出现在已打开的组合包上；Host 不寻址的行锁定
装	**添加插件** → 包名 / Git / 压缩包 / 本地路径	先 inspect 再 pnpm 安装；失败可重试/换源
配	本页"内置插件配置"分组：子智能体、网页搜索、Agent 循环、终端	配置页在插件页，不在设置里
卸	卸载并确认	随 DSH 发布的 bundle 只能关不能卸

风险与回退：改坏 `cordis.patch.yml` 可能让界面起不来——**改前先复制一份** `.bak`，出事就还原。装插件失败时 Host 会自动恢复 `package.json` 和锁文件，但你的手写配置不会自动恢复。

C4. 新手开关建议

- **第一天就用**：会话+工作区、右侧栏文件树/预览、`@` 与 `/`、交付/改动卡片、后台任务、定时提醒。
- **先把权限改回"工作区内修改"**：本机默认是完全权限且不弹审批，零基础用户不该在没有护栏时让 agent 跑命令。
- **想清楚再开**：插件市场（第三方代码）、会话搜索（模型侧搜历史）、会话标题变体。
- **先别碰**：自动审查（慢且贵）、Inspector（未脱敏）、Ralph/Worktree/思维链翻译、MCP、手改配置文件。

附录 D：界面、命令与快捷键速查

D1. 常用界面操作

你想做什么	去哪
新建会话	左侧栏"新会话"按钮，或在某个工作区行里新建
添加/切换工作区	左侧栏"添加工作区"；工作区行菜单可重命名、排序、分组
重命名 / 置顶 / 分支 / 归档会话	会话行"…"菜单（或悬停按钮）
搜索会话	`Mod+K`（侧栏搜索按钮）；默认只搜标题与工作区名
看 DSH 改了哪些文件	每轮回答末尾的"改动文件卡片"，点行看 diff
预览文件	右侧栏"新标签页 → 文件"，或 `Mod+P`（桌面）
开终端	右侧栏"新建终端"
开浏览器标签	右侧栏"浏览器"；`Ctrl+T`（Windows）
换模型/推理档	输入框模型入口，或 `/model`
换权限	输入框权限控件，或 `/permission`；通用设置里改"新会话默认"
管理插件	侧栏"插件"；设置 → 内置插件（只读）
看用量/余额	设置 → 账号 / 通用设置 → 性能与用量
计划模式开关	输入框计划入口 或 `/plan`；关闭 `/plan off`
中断生成	生成中出现的"停止"按钮；想补充就"插话"（`Ctrl/Cmd+Enter` 发送排队消息）

D2. 斜杠命令（本机可确认的）

命令	作用
`/plan`	进入计划模式（`/plan off` 退出）
`/model`	切模型 / 推理等级
`/permission`	切权限模式
`/compact`	手动压缩上下文
`/export`	导出本会话日志（ZIP）
`/goal`	管理会话目标
`/skill`	查看/加载技能
`/feedback`	提交反馈

> 命令菜单是运行时拼的，以你输入 `/` 时看到的为准。

D3. 快捷键（Windows 用 Ctrl，macOS 用 Cmd）

快捷键	作用
`Mod+K`	搜索会话（标题 + 正文，正文需开启会话搜索）
`Mod+P`	快速打开工作区文件（桌面端）
`Ctrl+T`	新建浏览器标签
`Ctrl/Cmd+Enter`	插话发送全部排队消息
`Ctrl/Cmd+Shift+.`	开关 NodeJS 诊断面板（Inspector）

附录 E：上下文与成本管理清单

E1. 先记住三个数字

数字	含义	对你的意义
1M token	本机上下文窗口	能装很多，但每步都要重发
约 80%	自动压缩触发线（≈ `min(窗口×80%, 窗口−输出上限−64K)`）	别拖到自动压缩，它本身要花一次大请求
约 2%	缓存命中价 / 未命中价比	"重读旧内容"便宜，但"每一步都背着大上下文"不便宜

参考价（`deepseek-flash`，元/百万 token，空闲时段）：缓存命中 0.02、缓存未命中 1、输出 4；高峰时段（工作日 9:00–12:00、14:00–18:00）翻倍。**以设置 → 账号里的实际用量为准。**

E2. 十二条省钱习惯（建议打印贴墙）

- ☐ 一个**可验收的子任务**一个会话，验收完就归档。
- ☐ 同一任务的**返工留在原会话**；换阶段先写 `handoff.md` 再新开。
- ☐ 别把长文贴进对话——给**文件路径**，让它切片读。
- ☐ 大文件先 `grep` / 读前 100 行，不要整份读进对话。
- ☐ 长输出让它**写文件**，对话里只留摘要。
- ☐ 上下文过 10–20 万就 `/compact` 或换会话，别等 80%。
- ☐ 重活排到**晚上/周末**（错峰半价）。

- ☐ 简单任务在模型菜单里**降推理档**。
- ☐ 只有难题才切更强的模型。
- ☐ 独立调研/批量任务用**子代理 / Workflow**，别开一堆长会话。
- ☐ **别频繁改 AGENTS.md**：改了会追加新指令，旧规则还在历史里。
- ☐ 每天收尾更新 `docs/progress.md`，让下一次会话能"接上"而不是"重讲"。

E3. 每周看一次用量

设置 → 账号 / 通用设置 → 性能与用量，重点看：

- 缓存命中率（越高越省）；
- 最长的几个会话（是不是该拆）；
- 高峰时段占比（能不能错峰）；
- 团队/子代理的花费占比（值不值）。

附录 F：常见错误急救

症状	最可能的原因	怎么修
界面打不开 / 一启动就报错	手改 `cordis.patch.yml` 写坏了	用备份的 `.bak` 还原；没有备份就删掉最近加的那几行
插件页关了东西，界面功能消失	关掉了界面插件	插件页重新打开；或还原 profile patch
改了配置不生效	需要重启，或行被上层覆盖	重开会话/应用；确认改的是 profile 的 patch
新会话里没有某个工具	它不在当前 agent 预设里	换预设（standard）或在预设编辑里加；这不是 bug
技能没出现	frontmatter 写错 / 目录层级错	`name` 用小写连字符；文件放 `.dsh/skills/<name>/SKILL.md`，只扫一层
AGENTS.md 没生效	工作目录不在项目根下 / 规则写得太抽象	确认 `git init` 的根；新开会话验证；把规则写成可执行句
AGENTS.md 改了但旧规则还在	更新是"追加消息"，旧文本仍在历史	正常现象；新会话验证；别频繁大改
它说"做完了"但文件不对	没验收	打开文件看；用"验收清单"返工
团队队友不动	inactive，或任务没派给它	`list_agents` 看状态 → `send_message` 唤醒；检查任务 owner

症状	最可能的原因	怎么修
`wait_agent` 立刻返回	没有活跃成员	先 `list_agents`，再用 `send_message` 唤醒需要的成员
任务 claim 失败 / 更新被拒	revision 过期	先 `team_task_get` 拿最新 revision 再改
子代理结果找不到	它只回复了，没写文件	要求"结果写进 output/xxx.md"
上下文已用很高、回答变笨	会话太长	`/compact`，或写交接后新开会话
账单比预期高	巨型会话 + 长输出 + 高峰时段	见附录 E 的十二条习惯
网页搜索总失败	网络/额度/服务	查网络与账号余额；换 `web_fetch` 直接抓 URL
Word/PDF 打不开或乱码	生成失败或字体问题	让它检查并重新生成；PDF 用 Office 技能导出
Pages 打开是 404	分支/目录设置不对	Settings → Pages 检查 `main` + 目录；等 1–3 分钟
误删/改坏了文件	无版本控制	有 git 就 `git restore`；没有就只能重做——**这就是每天 commit 的理由**

附录 G：术语中英对照

中文	英文	一句话解释
智能体	agent	能用工具干活的 AI
工作区	workspace	磁盘上一个项目目录
会话	session	一次任务的对话记录
上下文	context	模型每次请求看到的全部内容
上下文窗口	context window	一次能装下的 token 上限
压缩	compaction	把早期历史总结成摘要
工具调用	tool call	模型执行的一次操作
沙箱	sandbox	限制 agent 能改哪里的"笼子"
审批	approval	动手前等你点允许
提示词	prompt	你给模型的任务说明
系统提示词	system prompt	DSH 内置的底层规则（用户改不了）
预设 / 智能体预设	preset	一个会话能用哪些工具的组合
技能	skill	按需加载的操作规程（SKILL.md）
插件	plugin	装进 DSH 的功能包

中文	英文	一句话解释
组合包	bundle	一批插件的装配集合
配置文件补丁	profile patch	`cordis.patch.yml`，按 id 覆盖配置
子代理	subagent	独立上下文的临时外援
智能体团队	Agent Team	Lead + 队友 + 共享任务板的协作机制
队长	Lead	当前会话，团队的协调者
队友	teammate	有名字、有职责、持久存在的成员
共享任务板	task board	团队的任务清单（含依赖/owner/写入范围）
写入范围	write scopes	任务预计改哪些文件（只是提示）
工作流	workflow	用 JS 脚本批量编排子代理
MCP	Model Context Protocol	接外部工具服务器的协议
交付物	deliverable	明确声明交给用户的文件
计划模式	plan mode	只出计划不动手，等你批准
缓存命中	cache hit	重复前缀按低价计费
推理等级	reasoning effort	模型思考档位，越高越慢越贵

附录 H：14 天打卡表

天	主题	完成 ✓	我卡住的地方 / 备注
1	认识 DSH：工作区、会话、第一次真任务	☐	
2	Agent 怎么工作：工具、权限、上下文、成本	☐	
3	提示词基础：交办单六要素	☐	
4	提示词进阶：@文件、计划、反问、返工	☐	
5	AGENTS.md 与工作区结构	☐	
6	插件与技能（上）：Office 三件套	☐	
7	周复盘 + 个人主页上线	☐	
8	子代理：并行调研	☐	

天	主题	完成 ✓	我卡住的地方 / 备注
9	Agent Team（上）：2 人小组	☐	
10	Agent Team（下）：3 人协作 + 审查	☐	
11	Workflow：批量处理	☐	
12	插件进阶：市场、实验、自建技能	☐	
13	结营项目（上）：选题 + 组队	☐	
14	结营项目（下）：验收 + 发布 + 复盘	☐	

附录 I：换电脑 / 重装清单与资源

I1. 重装清单

1. 安装 DSH 桌面版（DeepSeek 官网下载），登录 DeepSeek 账号；
2. 恢复工作区：把 `E:\AI-Learning` 整个目录拷到新机器（含 `.git`）；
3. 安装 Git，配置用户名邮箱：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
git config --global init.defaultBranch main
```

4. 安装 GitHub CLI 并登录：`gh auth login`（HTTPS + 浏览器）；
5. 在 DSH 里重新注册工作区：侧栏 → 添加工作区 → 选 `E:\AI-Learning`；
6. 新建会话验证：AGENTS.md 是否生效、技能是否出现、`/export` 是否可用；
7. 重要配置备份：`C:\Users\你的用户名\.dsh\profiles\desktop\cordis.patch.yml`（手改过的配置）与 `~/.dsh/AGENTS.md`（全局规则）。

I2. 学习资源

本机（最贴近你）

- `E:\AI-Learning\DeepSeek-工作区整理指南.md`：工作区/会话/上下文/成本的真实数据分析；
- `E:\AI-Learning\工作区模板\`：AGENTS.md、docs/、tasks/、skills/ 的可复用模板；

- `E:\AI-Learning\GitHub 学习\14 天 GitHub 学习计划.md`：Git/GitHub 基础，和本计划互补；
- DSH 安装包内置的官方文档：每个插件包目录下的 `README.zh.md`（开发者视角，遇到具体功能再查）。

线上

- DeepSeek 官网与 DSH 下载/更新页（以官网为准）；
- DSH 开源仓库（`deepseek-ai/deepseek-harness`）：源码、issue、更新日志；
- DeepSeek API 文档：模型、价格、上下文缓存；
- GitHub Pages / Actions 官方文档（发布作品用）。

遇到问题怎么办

1. 先看报错原文和工具卡片里的实际命令；
2. 问 DSH：把「报错原文 + 你做了什么 + 期望结果」一起给它；
3. 查本机文档与官方仓库；
4. 还不懂就缩小问题（换最小例子复现）；
5. 最后才问人，贴完整信息。

结营的话

14 天能学会的只是"会用"。真正拉开差距的，是接下来这件事：**每周用 DSH 做一件真实的、可以给别人看的东西。**

会写提示词，你只是"会用 AI"；会拆任务、会验收、会让一群 AI 为你工作——那才是"会带 AI 干活"。祝你把这份能力用在真正想做的事上。

本计划基于 DSH 桌面版 0.2.1-rc.5（2026 年 11 月）编写；界面文案与功能可能随版本变化，以你实际看到的为准。